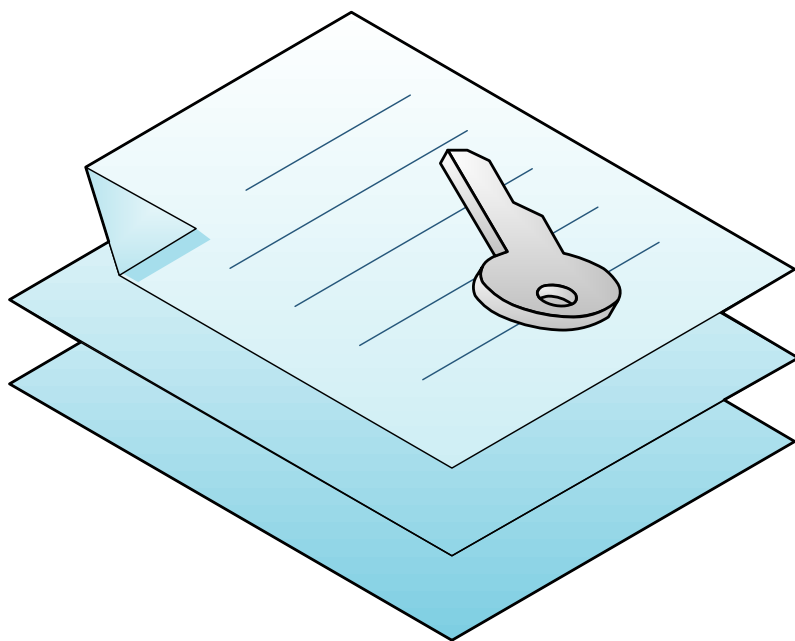


**Л. Г. АКУЛОВ
В. Ю. НАУМОВ**

ХРАНЕНИЕ И ЗАЩИТА КОМПЬЮТЕРНОЙ ИНФОРМАЦИИ



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РФ
ВОЛГОГРАДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Л. Г. Акулов, В. Ю. Наумов

ХРАНЕНИЕ И ЗАЩИТА КОМПЬЮТЕРНОЙ ИНФОРМАЦИИ

Учебное пособие



Волгоград
2015

УДК 681.31 (075)

Рецензенты:

кафедра «Теория и методика обучения математике и информатике»
Волгоградского государственного социально-педагогического университета,
зав. кафедрой д-р пед. наук, профессор *Т. К. Смыковская*;

доцент кафедры «Биотехнические системы и технологии»
Волгоградского государственного медицинского университета
канд. физ.-мат. наук *М. В. Петров*

Печатается по решению редакционно-издательского совета
Волгоградского государственного технического университета

Акулов, Л. Г.

Хранение и защита компьютерной информации : учеб. пособие /
Л. Г. Акулов, В. Ю. Наумов ; ВолгГТУ. – Волгоград, 2015. – 64 с.
ISBN 978–5–9948–1819–0

Рассмотрены базовые понятия информационной безопасности. Приведены основные классические шифры, наглядно показывающие суть процессов сокрытия информации.

Предназначено для студентов, изучающих курсы, связанные с основами информационной безопасности.

ISBN 978–5–9948–1819–0

© Волгоградский государственный
технический университет, 2015
© Л. Г. Акулов, В. Ю. Наумов, 2015

ОГЛАВЛЕНИЕ

Введение.....	4
1. Базовые понятия теории информационной безопасности.....	7
1.1. Основные определения.....	7
1.2. Общие принципы симметричного шифрования.....	11
2. Простейшие симметричные шифры.....	17
2.1. Простейшие шифры подстановки.....	17
2.2. Простейшие шифры перестановки.....	30
3. Блочные и поточные шифры.....	35
3.1. Блочные шифры.....	35
3.2. Поточные шифры.....	37
3.3. Режимы комбинации шифрования блоков.....	38
4. Современные симметричные шифры потока.....	49
4.1. Алгоритм RC4.....	49
4.2. Алгоритмы A5 как база шифрования в сетях GSM.....	52
Рекомендуемая и используемая литература.....	61

Введение

Сегодня уровень информационного окружения человека велик, как никогда ранее. То, что происходит в связи с вхождением информационно-коммуникационных технологий в повседневную жизнь иначе как революцией не называют, на основании этого вводят понятие *информационного общества*. Более того, революционные процессы не останавливаются ни на миг, и приходится прибегать к новым понятиям типа *информационное общество 2.0*, с такими темпами не за горами 3.0 и т.д. Однако как отдельный человек, так и общество в целом весьма скупо понимает суть информационных процессов, несмотря на то, что они оказывают прямое, порой довлеющее влияние на большинство процессов вокруг. Банковские операции, электронная переписка, социальные сети, связь, системы управления городом, зданием, транспортными потоками и отдельными самолетами или автомобилями (и т. д.) – везде огромный поток информации, требующий грамотного использования. В руках злоумышленника таковые потоки могут являться грозным оружием.

Цель настоящего учебного пособия дать основы в области информационной безопасности, на примитивных примерах объяснить ключевые принципы, лежащие в фундаменте этой области знаний.

Для умелого использования средств защиты информации необходимо знание теории вероятностей, методов и средств программирования, теории чисел, дискретного анализа, знакомства с понятиями алгоритмической и общей сложности. Информационная безопасность в общем случае является наукой секретной, поскольку неразрывно связана с военной и экономической безопасностью различных субъектов, начиная от отдельного человека, заканчивая государственными образованиями. Как следствие, в методологической основе информационной безопасности должна лежать

теория игр, рассматривающая законы взаимодействия и противостояния нескольких участвующих в игре субъектов. В теории игр основной является оптимизация некоего функционала, называемого выигрышем. Как следствие, для умелого применения средств защиты информации нужны знания в области раздела математики, называемого оптимизацией. Современные методы построения безопасных систем требуют знания всех разделов физики, включая квантовую механику (есть даже активно развивающееся направление, называемое *квантовой криптографией*). Наконец, в самом общем случае конечным потребителем информации является человек, а потому необходимо изучение его социально-биологической сущности. Ввиду важности рассматриваемой темы отдельные ее части стандартизованы и регламентируются законодательно. Для этого применяются специальные юридические нормы и правила, которые можно объединить в категорию нормативно-правовых актов в области информационной безопасности, или в более общем случае – *информационного права*. Основным предметом регулирования информационного права выступают информационные отношения, то есть общественные отношения в информационной сфере, возникающие при протекании информационных процессов – процессов производства, сбора, обработки, накопления, хранения, поиска, передачи, распространения и потребления информации.

Естественно, что охватить с достаточной подробностью все указанные выше области в рамках одного учебного пособия задача, на наш взгляд, невыполнимая, именно потому выбрана наиболее полезная и непротиворечивая область в рамках защиты информации. Таковой областью является практическая *криптография*, поскольку на формальном языке математики описывает механизмы информационной безопасности. Криптография – это наука о шифрах. Дословно можно перевести это понятие как *тайнопись*.

Криптография является бурно развивающейся областью знаний, вклад в которую вносят ученые из разных государств. Международным языком общения является английский, а потому для большинства терминов в данном учебном пособии приводятся их англоязычные аналоги. Многие понятия достаточно молоды или служат основой для иных определений, как следствие, знание их написания на английском необходимо для сохранения возможности оперативного получения новых знаний по изучаемой теме.

Данное небольшое пособие задумывалось как первая часть в череде учебных материалов по информационной безопасности, а потому дает лишь начальные понятия. В частности, здесь не рассмотрены вопросы современных блочных шифров с симметричными ключами, системы шифрования с открытым ключом, цифровые подписи и комплексные протоколы защищенного информационного обмена. Отдельной работой следует издать задачник, позволяющий закрепить весь комплекс полученных знаний на практике. Все это является полем для дальнейшей работы.

1. Базовые понятия теории информационной безопасности

1.1 . Основные определения

Цели обеспечения информационной безопасности заключаются в обеспечении: конфиденциальности (confidence), целостности (integrity) и доступности (availability). Иногда к ним добавляют подотчётность (accountability), достоверность (reliability), аутентичность или подлинность (authenticity) и неотрекаемость (non-repudiation).

Криптография, как подраздел информационной безопасности, обеспечивает достижение следующих целей:

конфиденциальность: защита данных или личной информации пользователя от несанкционированного просмотра;

целостность данных: защита данных от несанкционированного изменения;

проверка подлинности: проверка того, что данные исходят действительно от определенного лица;

неотрекаемость: ни одна сторона не должна иметь возможность отрицать факт отправки сообщения.

Для возможности описания современных криптографических методов следует рассмотреть базовые категории теории шифрования, так называемые **криптографические примитивы** (cryptographic primitives). Стандартный набор примитивов следующий:

шифрование с закрытым ключом (симметричное шифрование). Secret-key encryption (symmetric cryptography). Преобразование данных с целью предотвращения их просмотра третьей стороной. Для шифрования и расшифровки используется один общий закрытый ключ. Принято выделять потоковые (stream) и блочные (block) шифры;

шифрование с открытым ключом (асимметричное шифрование) Public-key encryption (asymmetric cryptography). Осуществляет преобразование данных с целью предотвращения их просмотра третьей стороной. В данном способе шифрования для шифрования и расшифровки используется набор, состоящий из открытого и закрытого ключей;

аутентификация сообщений или **имитовставка** (message authentication code, MAC – код аутентичности сообщения). Делится на две части: 1) **криптографическая подпись**. Удостоверяет, что данные действительно исходят от конкретного лица, используя уникальную цифровую подпись этого лица; 2) проверка целостности сообщения через **криптографическое хэширование**. Отображение данных любого размера в байтовую последовательность фиксированной длины. Результаты хэширования статистически уникальны; отличающаяся хотя бы одним байтом последовательность не будет преобразована в то же самое значение (в идеале изменение одного бита на входе приводит к изменению ровно половины бит на выходе).

На основе базовых примитивов строятся правила их использования, называемые протоколами. Наиболее важными в криптографии являются:

выработка ключа;

отправка и прием сообщения.

Для того чтобы шифрование информации обеспечивалось с заданным уровнем надежности, необходимо следовать так называемому принципу **Огюста Керкхофса** (Auguste Kerckhoffs). Стойкость криптосистемы должна определяться только секретностью ключа шифрования. Система не должна быть секретной, и если она попадет в руки противника, это не должно причинить неудобства. Часто этот принцип называют отсутствием **шифрования через запутывание**. Действительно, открытые и полностью изученные методы дают математически доказуемый уровень информаци-

онной защиты, в отличие от наспех придуманных и, практически всегда, обладающих серьезной уязвимостью собственных методов.

Для осуществления информационного обмена применяется выработанная система фиксации этой информации на определенных физических или логических носителях. Конечной целью этого обмена является обмен знаниями. Такого рода представления подразумевает наличие у них следующих свойств:

синтаксис – правила записи на информационный носитель;

семантика – смысл того, что написано;

прагматика – полезность информации для того, кто ей обладает.

Обеспечение информационного обмена подразумевает целый ряд субъектов и объектов, принимающих в нем участие. При этом субъекты могут состоять между собой в различных отношениях (дружественные, нейтральные и враждебные) в разные моменты времени в разных условиях. Все эти субъекты могут обладать различными способностями и разными личными интересами в условиях информационного обмена. Число участников информационного обмена, вообще говоря, не ограничено. Рассмотреть все возможные сочетания взаимодействия указанных субъектов не представляется возможным. Однако есть типовые ситуации сочетаний, встречающиеся достаточно часто на практике. Для этих ситуаций принята определенная терминология, часто граничащая с «криптографическим жаргоном» или «фольклором», но она распространена повсеместно и позволяет достаточно просто описывать те или иные ситуации и явления.

Типовые участники информационного обмена описаны ниже.

Алиса и *Боб* (Alice and Bob) – основные участники информационного обмена. Как правило, Алиса хочет послать сообщение Бобу.

Чарли (Charlie) или *Кэрол* (Carol) – третий участник соединения.

Дейв (Dave) – четвертый участник (и так далее по латинскому алфавиту).

Ева (Eve) – пассивный злоумышленник, который может прослушивать сообщения между Алисой и Бобом, но не может влиять на них.

Мэллори (Mallory) – активный злоумышленник, который, в отличие от Евы, может изменять сообщения. Сложность защиты от Мэллори выше чем от Евы.

Крейг (Craig) – взломщик паролей (обычно встречается в ситуации с хранимыми хэшами).

Оскар (Oscar) – имеет возможности, аналогичные Мэллори, но при этом не является злоумышленником. Его цель – исследование системы безопасности.

Сайбил (Sybil) – злоумышленник, который создает множество фиктивных пользователей в информационной системе с целью нанесения урона репутации прочих пользователей.

Трент (Trent), доверенный арбитр – своего рода нейтральная третья сторона, чья точная роль изменяется в зависимости от стадии обсуждения протокола.

Уолтер (Walter) – надзиратель, может быть необходим для охраны Алисы и Боба, в зависимости от обсуждаемого протокола.

Вэнди (Wendy) от слова «whistleblower» – доносчик, является инсайдером с привилегированным доступом, который может быть в состоянии обнародовать информацию.

Естественно, что здесь указаны далеко не все возможные роли участников информационного обмена.

Отдельно следует сказать о таком явлении как **хакерство**. **Хакер** или компьютерный взломщик – это участник информационного обмена, занимающийся поиском уязвимостей в системах передачи информации. В зависимости от конечных намерений хакеров принято делить на «черных», использующих найденные уязвимости в корыстных целях и «белых», вы-

являющих недостатки информационных систем с целью последующего избавления от этих недостатков.

Среди основных формализмов, применяемых в математических выражениях в процессе шифрования, следует выделить ряд устоявшихся обозначений (табл. 1.1). При этом M и P часто используются как понятия-синонимы.

Таблица 1.1

Категория	Обозначение	Этимология
Шифр	c	Cipher
Ключ	k	Key
Сообщение	m	Message
Открытый текст	p	Plaintext
Функция шифрования	$E(...)$	Encryption function
Функция расшифровки	$D(...)$	Decryption function

1.2. Общие принципы симметричного шифрования

Если Алиса имеет сообщение E , предназначенное для передачи по незащищенному каналу Бобу и они оба обладают некоторым секретом k , то возможно построение функции **шифрования**:

$$c = E(m, k). \quad (1.1)$$

Чтобы расшифровывать сообщение должна существовать однозначная обратная функция **дешифровки**:

$$m = D(c, k). \quad (1.2)$$

Функции D и E обладают свойством взаимобратимости:

$$\begin{cases} m = D(E(m, k), k), \\ c = E(D(c, k), k). \end{cases} \quad (1.3)$$

Это свойство называется **биекцией** или взаимной однозначностью.

Схема информационного обмена с закрытым ключом показана на рисунке 1.1. Здесь в отдельную категорию выделен ключ. На самом деле, выработка надежного ключа для шифрования – важнейшая задача криптографии, которую пока рассматривать не будем. В идеальном случае ключ должен обладать свойствами абсолютной случайности, но это недостижимо. Потому вырабатываемый ключ всегда обладает определенной мерой ненадежности.

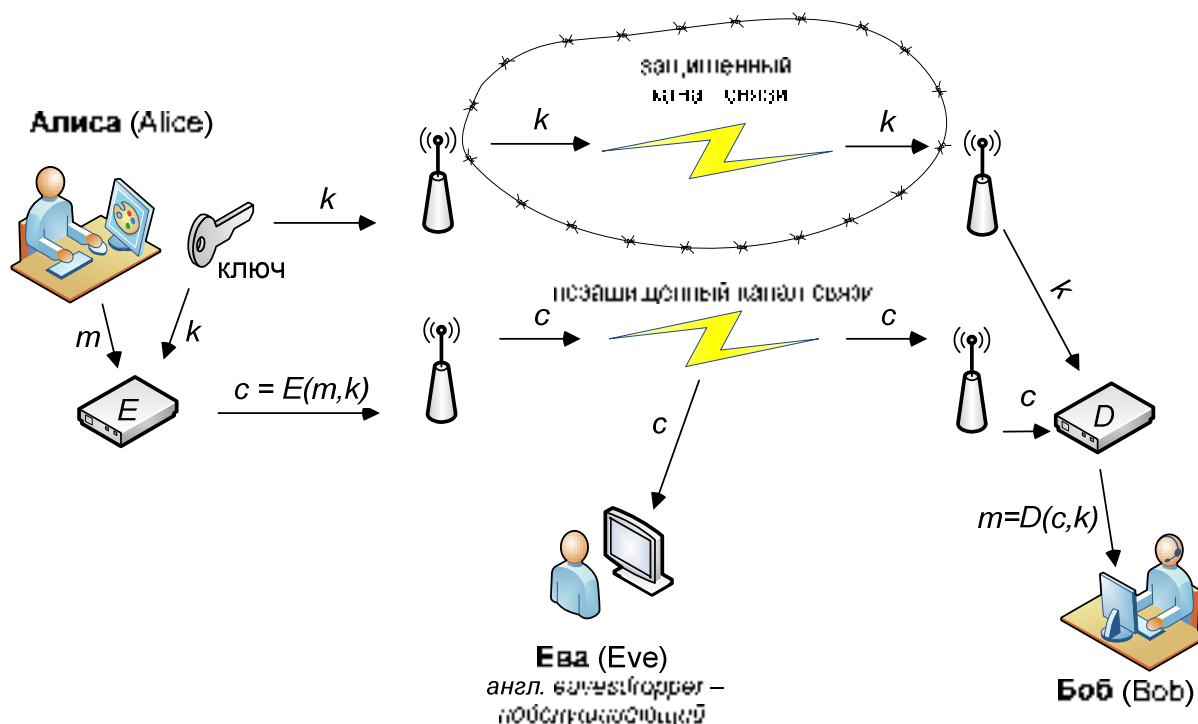


Рис. 1.1. Схема информационного обмена с закрытым ключом

Целью применения схемы, показанной на рисунке 1.1, является достижение конфиденциальности. Никто посторонний не должен получить семантической составляющей сообщения. Роль постороннего участника играет Ева.

Главной проблемой симметричных систем является необходимость обмена ключами по защищенному каналу. Способ организации такого рода канала является отдельной темой теории информационной безопасности.

Шифрование с закрытым ключом весьма быстрое (в сравнении с шифрованием с открытым ключом), хорошо преобразует большие массивы информации. Асимметричное шифрование, имеет математические ограничения на объем шифруемых данных.

Множества, на которых определены функции (1.1) и (1.2) следующие: M – множество возможных сообщений; K – множество возможных ключей; C – множество возможных шифров. Для множеств определены отображения:

$$\begin{cases} E: M \times K \rightarrow C, \\ D: C \times K \rightarrow M. \end{cases} \quad (1.4)$$

Важную роль играют мощности множеств M , K и C . Допустим, что для мощностей множеств сообщений выполнено

$$|M| = |C|. \quad (1.5)$$

Это позволяет нам определить понятие *абсолютной криптографической стойкости* (perfect secrecy). Термин предложен Клодом Шенноном в работе Communication Theory of Secrecy Systems. Ева получает зашифрованное сообщение, но не получает никакой дополнительной информации об исходном сообщении. Здесь $\forall m_0, m_1 \in M$ и $\forall c \in C$, вероятность того, что это c получено из m_0 равна вероятности того, что c получено из m_1 .

Перебрав всё множество K можно вычислить все возможные сообщения $D(c, k)$ для конкретного c . Если $|K| < |M|$, то в результате Ева сузит круг возможных сообщений, то есть получит дополнительную информацию. Следовательно, стойкость не будет абсолютной.

Можно привести следующий пример. Пусть Алиса использует ключ k в 128 бит ($|K| = 2^{128}$) для шифровки сообщения m размером 1 КБ. Очевидно, что получив 128-битный ключ, Ева полностью восстановит сообщение m , то есть ей не хватает 128 бит. А изначально ей не хватало $8 \times 1 \times 1024 = 81928$ бит. В результате перехвата она узнала объем информации, составляющий $(1 - 128/81928) = 0,984375$ от того, что за-

шифровано. То есть более чем 98% , но при этом семантика сообщения для Евы практически нулевая (Ева ничего не поняла)! Почему это так? Причина – мощность множества всех возможных исходных сообщений, которая астрономически огромная. В то время, как мощность ключа просто огромная. Теоретически, можно применить эвристики для вскрытия исходного сообщения. Это и есть реализация *атаки* на шифр или *криптоанализ*.

Вывод такой: для достижения абсолютной стойкости размер ключа должен быть не меньше, чем размер сообщения:

$$|M| \leq |K|. \quad (1.6)$$

Следующая особенность такова. Если применяется детерминированная функция E с одним и тем же ключом k дважды, то при одинаковых входных сообщениях m Ева дважды получит одинаковую шифровку c . Это уже потеря информации, в отдельных случаях вплоть до полной утраты конфиденциальности.

Если противником будет Мелори, то имея возможность воздействия на сообщения, получаемые Бобом, при обладании двумя разными информационными блоками, зашифрованными одним ключом возможна прямая подмена без вскрытия этого ключа. То есть нарушается целостность передаваемой информации.

Для решения проблемы повторного ключа вводят понятие так называемого **одноразового блокнота** (one-time pad, OTP). Если исходное сообщение m – это строка бит (0 и 1), то ключ k – это случайная строка бит той же длины. Другими словами:

$$m \in \{0,1\}^{|m|}, k \in \{0,1\}^{|k|} \text{ и } |k| = |m|. \quad (1.7)$$

Функция шифрования E осуществляет выполнении побитового XOR сообщения с ключом дешифрование аналогично. Если вероятность выбора каждого ключа одинакова, то по зашифрованному сообщению ничего

нельзя сказать об исходном. Исходное может быть любым с одинаковой вероятностью. Для записи ключа и нужен одноразовый блокнот.

Блокнот одноразовый по двум причинам:

- иначе можно будет опознать одинаковые сообщения;
- если к шифротекстам, полученным одним ключом применить побитовый XOR, то ключи обнулятся и получится побитовая разность исходных сообщений.

Остановимся на описании операции побитового «исключающего ИЛИ» (XOR), которое обозначают как « $\oplus$ ». Напомним, что свойства задаются таблицей истинности:

$$\begin{cases} 0 \oplus 0 = 0, \\ 0 \oplus 1 = 1, \\ 1 \oplus 0 = 1, \\ 1 \oplus 1 = 0. \end{cases} \quad (1.8)$$

Оператор XOR очень активно используется в криптографии в силу того, что он биективен и позволяет манипулировать отдельными битами сообщения. Действительно,

$$\begin{cases} m \oplus k = c, \\ c \oplus k = m \oplus k \oplus k = m. \end{cases} \quad (1.9)$$

Для формирования эффективного ключа он должен быть абсолютно случайным, то есть если ключ состоит из нулей и единиц, то появление в каждом его бите как «1», так и «0» равновероятно. Однако на практике применяют *псевдослучайные генераторы* (pseudo random generators, PRG).

Это значит, что для некоторого набора ключей небольшого размера, принадлежащих множеству K_0 , существует функция $PRG: K_0 \rightarrow \{0,1\}^n$, которая генерирует k – последовательность бит длиной n такую, что «эффективная Ева» не сможет отличить ее от истинно случайной. При этом $k_0 \in K_0$ и $|k_0| < |k| = n$. Значит, PRG можно использовать для генерации ОTR.

Что значит «эффективная Ева»? Это значит, что ее возможности в плане вычислительных мощностей не безграничны. Алгоритмы по расшифровке работают полиномиальное время от $\log(|K_0|)$.

Истинно случайной будет такая последовательность бит, в которой число угадываний последующего бита точно равно числу промахов при попытках воссоздать некоторую их последовательность в пределе бесконечной длины. С точки зрения теории вероятностей при последовательностях конечной длины это значит, что

$$P(A|B) - P(A|C) < \varepsilon. \quad (1.10)$$

Здесь $P(x|y)$ означает условную вероятность события x при выполнении y . Событие A – «считаем, что строка случайна», событие B – «строка случайна», событие C – «строка сгенерирована PRG ». Параметр ε есть параметр случайности. Вообще говоря, чем он меньше, тем лучше работает PRG при выполнении условия (1.10). С практической точки зрения параметр ε можно считать приемлемым, если он имеет экспоненциальный спад в зависимости от мощности базового множества возможных ключей K_0 :

$$\varepsilon(K_0) \sim \log(|K_0|). \quad (1.11)$$

При наличии такой функции PRG ее выход может быть использован как ключ к ОTR.

2. Простейшие симметричные шифры

2.1. Простейшие шифры подстановки

Есть виды шифров, которые имеют лишь исторический интерес, поскольку их применение в современных системах не принесет приемлемого уровня защиты. Тем не менее, знакомство с такими шифрами полезно, поскольку они достаточно просты и помогают понять общие криптографические принципы как в теории, так и на практике. Кроме того, знание этих шифров и методов их компрометации позволяет избавиться от соблазна их применения в собственных информационных системах. К базовым понятиям шифров можно отнести шифры: 1) **подстановки** (замена одних значений на другие по индексной таблице, замене подвергаются группы элементов шифруемого блока); 2) **перестановки** (перестановка структурных элементов шифруемого блока данных); 3) **функциональных преобразований** (выполнение сдвигов, логических и арифметических операций над элементами данных).

Табличная подстановка. Это такой вид шифра, в котором каждому символу исходного алфавита ставится в соответствие другой символ из этого алфавита. Например букву «а» меняем на «д», «д» меняем на «ц», «ц» меняем на «я» и так далее, пока не будет выбран весь алфавит. Мощности возможного множества перестановок (ключей) $n!$, где n – это мощность алфавита. Действительно, если взять за основу алфавит русского языка в котором 33 буквы, то первую букву можно закодировать 33-мя способами, вторую 32-мя, третью 31-м и так далее.

Пусть ключ k задан таблицей 2.1.

Текст m = «ПРИШЕЛУВИДЕЛПОБЕДИЛ»,

Шифр c = «ЯЖТВЦЦЦЁТНЦЬЯЭАЩНТЬ»

Таблица 2.1

<i>а</i>	<i>б</i>	<i>в</i>	<i>г</i>	<i>д</i>	<i>е</i>	<i>ё</i>	<i>ж</i>	<i>з</i>	<i>и</i>	<i>й</i>
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
<i>м</i>	<i>а</i>	<i>ё</i>	<i>ь</i>	<i>н</i>	<i>щ</i>	<i>д</i>	<i>ы</i>	<i>е</i>	<i>т</i>	<i>с</i>
<i>к</i>	<i>л</i>	<i>м</i>	<i>н</i>	<i>о</i>	<i>п</i>	<i>р</i>	<i>с</i>	<i>т</i>	<i>у</i>	<i>ф</i>
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
<i>г</i>	<i>ь</i>	<i>ч</i>	<i>б</i>	<i>э</i>	<i>я</i>	<i>ж</i>	<i>х</i>	<i>з</i>	<i>ц</i>	<i>ю</i>
<i>х</i>	<i>ц</i>	<i>ч</i>	<i>ш</i>	<i>щ</i>	<i>ь</i>	<i>ы</i>	<i>ъ</i>	<i>э</i>	<i>ю</i>	<i>я</i>
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
<i>л</i>	<i>ш</i>	<i>й</i>	<i>в</i>	<i>у</i>	<i>к</i>	<i>ф</i>	<i>о</i>	<i>и</i>	<i>р</i>	<i>п</i>

Для русского алфавита получим максимальную длину ключа $|k| = \log_2(33!) \approx 123$ бита. Что на первый взгляд может показаться приемлемым. Однако этот шифр не выдерживает атаки по частотам, поскольку известно, что частота встречаемости букв в текстах неравномерна.

Таблица 2.2

Ранг	Буква	Частота	Ранг	Буква	Частота	Ранг	Буква	Частота
1	о	0,10983	12	м	0,03203	23	й	0,01208
2	е	0,08483	13	д	0,02977	24	х	0,00966
3	а	0,07998	14	п	0,02804	25	ж	0,0094
4	и	0,07367	15	у	0,02615	26	ш	0,00718
5	н	0,067	16	я	0,02001	27	ю	0,00639
6	т	0,06318	17	ы	0,01898	28	ц	0,00486
7	с	0,05473	18	ь	0,01735	29	щ	0,00361
8	р	0,04746	19	г	0,01687	30	э	0,00331
9	в	0,04533	20	з	0,01641	31	ф	0,00267
10	л	0,04343	21	б	0,01592	32	ъ	0,00037
11	к	0,03486	22	ч	0,0145	33	ё	0,00013

Как следствие, имея достаточно длинный шифр, мы будем стремиться к обнажению статистических особенностей языка. Таблица 2.2 является буквенным паттерном, к которому стремятся тексты на русском языке.

Кроме того, в языке много особенностей, составления букв в последовательности. Например, слова не начинаются с «ъ» или «ь», «ь» всегда идет следом за согласной и так далее. При знании и применении всех этих особенностей задача по вскрытию простых шифров подстановки становится тривиальной.

Однако шифр простой табличной подстановки можно модифицировать, если произвести выравнивание частот символов. Суть модификации состоит в том, что для наиболее часто встречающихся символов применяются несколько кодов. В результате каждый из этих подкодов имеет частоту встречаемости примерно равную самому редкому символу. При этом таблица кодов становится намного шире, чем базовый алфавит. Например, если сравнивать буквы «о» и «ё» из таблицы 2.2, то видно, что их частота отличается почти в 1000 раз. А это значит, что только для буквы «о» придется резервировать 1000 символов. Это неприемлемо. Потому на практике, применяли более мелкие кратности, например 5-7 для наиболее часто встречающихся букв. Немного увеличивая размер ключа, добивались существенно большей однородности в шифре, что на много порядков увеличивало стойкость к частотным атакам.

Шифр Цезаря. Еще более простой реализацией шифра подстановки является так называемый шифр Цезаря. Это такой вид шифра, при котором для подстановки выбирается всего одна пара букв, а все прочие подстановки вычисляются на ее основе. Если алфавит имеет порядок, то каждой его букве ставится в соответствие натуральное число, кодирующее номер этой буквы в алфавите. Для кодирования берется произвольная пара букв x_i и x_j . Ключ вычисляется как

$$k = [ord(x_i) - ord(x_j)] \bmod N, \quad (2.1)$$

где N – мощность алфавита. Для русского языка $N = 33$, для английского $N = 26$. $ord(\dots)$ – функция взятия номера буквы в алфавите, $\bmod$ – оператор нахождения остатка от деления. Про ключ k говорят, что он находится в кольце $\mathbb{Z}_N$. Мощность множества всех ключей равна N .

Таким образом, стойкость шифра Цезаря практически нулевая, поскольку позволяет осуществлять вскрытие полным перебором, не прибегая к вычислительной технике.

Таблица 2.3 содержит пример, построенный с ключом, равным «3».

Таблица 2.3

а-01	б-02	в-03	г-04	д-05	е-06	ё-07	ж-08	з-09	и-10	й-11
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
э-31	ю-32	я-33	а-01	б-02	в-03	г-04	д-05	е-06	ё-07	ж-08
к-12	л-13	м-14	н-15	о-16	п-17	р-18	с-19	т-20	у-21	ф-22
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
з-09	и-10	й-11	к-12	л-13	м-14	н-15	о-16	п-17	р-18	с-19
х-23	ц-24	ч-25	ш-26	щ-27	ь-28	ы-29	ъ-30	э-31	ю-32	я-33
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
т-20	у-21	ф-22	х-23	ц-24	ч-25	ш-26	щ-27	ь-28	ы-29	ъ-30

Применим таблицу для шифровки:

m = «ПРИШЕЛУВИДЕЛПОБЕДИЛ»

Получится шифр c = «МНЁХВИРЯЁБВИМЛЮВБЁИ».

Шифр Виженера. Представляет собой один из наиболее известных шифров подстановки, его элементы лежали в основе работы знаменитой шифровальной машины Энигма, используемой нацистами во время Второй мировой войны. Сам шифр очень простой и представляет собой, по сути,

блочный шифр Цезаря. Здесь в качестве ключа k берется некоторое слово, составленное из символов базового алфавита. Соответственно, текст m разбивается на $|k|$ блоков. Для каждого символа из блока применяется смещение аналогичное шифру Цезаря.

Для кодирования блока $m_{i,j}$ берется символ $k_i, i \in 1..|k|$. Здесь

$$\begin{aligned} k &= k_1 \parallel k_2 \dots \parallel k_{|k|}, \\ m_j &= m_{1,j} \parallel m_{2,j} \dots \parallel m_{|k|,j}, \\ m &= m_1 \parallel m_2 \dots \parallel m_{\lfloor |m|/|k| \rfloor}. \end{aligned} \quad (2.2)$$

Оператор « $\parallel$ » есть оператор конкатенации, то есть «склеивания» символов. Например «ваава» $\parallel$ «уууу» = «ваавауууу», «р» $\parallel$ «к» = «рк» и т.п. Смещение по символам осуществляется путем вычисления

$$c_{i,j}^* = 1 + ([ord(k_i) + ord(m_{i,j}) - 2] \bmod N), \quad (2.3)$$

где N – мощность алфавита. Элемент шифра $c_{i,j}^*$ находится в кольце $\mathbb{Z}_N$. Смещения « -2 » и « $+1$ » возникают вследствие счета букв с «1», а не с нуля, как того требует модульная арифметика.

Закодируем при помощи составленной таблицы фразу $m =$ «ПРИШЕЛУВИДЕЛПОБЕДИЛ» ключом $k =$ «ЦЕЗАРЬ».

Получим таблицу 2.4:

Таблица 2.4

m	П	Р	И	Ш	Е	Л	У	В	И	Д	Е	Л	П	О	Б	Е	Д	И	Л
m^*	17	18	10	26	6	13	21	4	10	5	6	13	17	16	2	6	5	10	13
k	Ц	Е	З	А	Р	Ь	Ц	Е	З	А	Р	Ь	Ц	Е	З	А	Р	Ь	Ц
k^*	24	6	9	1	18	30	24	6	9	1	18	30	24	6	9	1	18	30	24
c^*	7	23	18	26	23	9	11	9	18	5	23	9	7	21	10	6	22	6	3
c	Ё	Х	Р	Ш	Х	З	К	З	Р	Д	Х	З	Ё	У	И	Е	Ф	Е	В

Результирующая шифровка $c =$ «ЁХРШХЗКЗРДХЗЁУИЕФЕВ».

В таблице приведены строки, в которых, помимо сообщения, ключа и шифра содержатся номера букв алфавита, а также их суммы, рассчитанные по формуле (2.3). Этим суммам и соответствует шифр.

Если мы хотим кодировать пробелы, то следует расширять алфавит и таблица для шифрования будет гораздо больше. При написании программ, автоматически реализующих шифр Виженера, следует учитывать что буква «Ё» может располагаться вне основной таблицы символов в стандартных компьютерных кодировках. Потому кодирование можно осуществлять без ее участия.

Можно поступить иначе, воспользовавшись таблицей смещений 2.5, которая позволяет производить кодирование вручную с визуальным контролем. Подобные таблицы чеканились на роторных барабанах машин осуществляющей кодирование механическим способом.

В представленной таблице нужно взять символ исходного сообщения m и найти его в первой буквенной строчке таблицы (совпадает с алфавитом без смещений), далее найти в столбце соответствующий символ ключа k . Пересечение строки и столбца даст необходимый символ шифра s . Эту процедуру повторить для всех символов сообщения m .

Осуществление дешифровки осуществляется аналогично. Для числового представления:

$$m_{i,j}^* = ([ord(c_{i,j}) - ord(k_i)] mod N) + 1. \quad (2.4)$$

Чтобы обратить шифр Виженера по таблице 2.5, следует символы шифра s искать в строках, соответствующих символам ключа k . Далее по координатам столбца восстанавливается исходный текст m .

Криптостойкость шифра Виженера выше, чем у шифра Цезаря. Но он обладает существенным недостатком в виде повтора применения ключа к блокам. Если знать длину блока, то становятся доступными взаимные разности между символами исходного сообщения. Поскольку вариантов

смещений всего N , то возможна атака перебора всех сочетаний. Кроме того становится доступной атака по таблице частот.

Таблица 2.5

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33
1	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я
2	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а
3	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б
4	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в
5	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г
6	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д
7	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е
8	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё
9	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж
10	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з
11	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и
12	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й
13	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к
14	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л
15	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м
16	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н
17	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о
18	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п
19	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р
20	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с
21	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т
22	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у
23	х	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф
24	ц	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х
25	ч	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц
26	ш	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч
27	щ	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш
28	ъ	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ
29	ы	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ
30	ь	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы
31	э	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь
32	ю	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э
33	я	а	б	в	г	д	е	ё	ж	з	и	й	к	л	м	н	о	п	р	с	т	у	ф	х	ц	ч	ш	щ	ъ	ы	ь	э	ю

Шифр четырех квадратов. Строят 4 квадрата размера 6х6 или 5х5 (в зависимости от объема алфавита). В каждый из квадратов случайным образом записывают алфавит.

Для того, чтобы зашифровать сообщение, последовательно выбирают по 2 буквы из него. Далее находят их, соответственно, в левом верхнем и правом нижнем квадрате. Комплементарно этим буквам выбирают другие 2 буквы, формирующие с первыми прямоугольник. То есть все четыре

буквы лежат в вершинах прямоугольника. Полученную пару букв (слева направо) помещают в шифр.

Пример сообщения: «*m* = «ПРИШЕЛ УВИДЕЛ ПОБЕДИЛ!»

(«пр иш ел _у ви де л_ по бе ди л!»).

Шифр: «БФЮЖБ!СГЁЯАНЬСБЫ,ЕБЯЦЩ»

(бф юж б! сг ёя ан ьс бы ,е бя цщ).

Таблица 2.6

ф	у	_	г	ю	ъ	о	ч	г	ш	ю	м
ь	а	в	к	д	э	_	и	я	д	н	э
!	ч	б	л	е	м	с	ц	з	у	е	!
х	й	ы	ё	т	н	п	а	щ	х	ь	ё
ц	ш	ж	и	о	с	т	б	й	ж	л	,
щ	,	з	я	п	р	р	в	к	ф	ы	ъ
н	я	л	ю	ц	з	э	ё	я	ш	ю	щ
_	д	,	х	а	ч	й	в	ц	ч	е	б
о	г	е	!	к	и	,	ж	д	з	х	ъ
м	у	с	ж	ы	ъ	ы	м	у	с	к	г
п	ф	ё	щ	б	ш	ф	!	и	р	о	л
р	в	т	ь	э	й	_	а	п	т	н	ь

Ключом к этому шифру является таблица 2.6.. Вариантов заполнения таблицы $n \times n$ будет всего $n^2! \cdot n^2! \cdot n^2! \cdot n^2! = n^{2!4}$. Для $n = 6$ получим мощность множества кодов $|K| = \log_2(36!^2) \simeq 553$ бита. Стойкость по ключу весьма неплохая, однако факт использования пар подряд идущих символов позволяет применить взлом шифра через частотные особенности языка (распространенность сочетания пар символов). Такая атака весьма эффективна при большом объеме шифротекста.

Биграммный шифр. Под этим шифром понимается кодирование не отдельных символов, а их пар. Соответственно, составляется ключевая таблица k размерностью $N \times N$, где N – мощность алфавита. В таблицу произвольным образом записываются все возможные пары буквосочетаний. Строки и столбцы проиндексированы упорядоченными символами алфавита. Для применения буквосочетания следует взять за элемент кода пару символов, индексирующих строку и столбец, в котором находится нужное буквосочетание (как пример – таблица 2.7).

Таблица 2.7

	а	б	в	...	я
а	ам	ро	цф	...	...
б	кс	нт	чя	...	...
в	жа	дз	яы	...	...
...	...	...	...	...	...
ю	...	...	...	...	кж
я	кц	...	...	...	пй

Так, например, слово $m =$ «баба» будет зашифровано как $s =$ «кскс».

Мощность ключа для этого шифра требуется очень большая. Для русского алфавита $|k| = 33 \times 33 = 1089$ пар символов. По перестановкам получается астрономическое число $1089!$. Но, к сожалению, этот шифр не устойчив к частотному анализу, поскольку не скрывает паросочетания символов. Аналогично можно разработать триграммный вообще n -граммный шифр. Эти шифры более криптостойки, но сложнее для реализации и требуют огромного объема ключевой информации. Все n -граммные шифры вскрываются с помощью частотного криптоанализа, опираясь на статистические таблицы встречаемости сочетаний n символов.

Алгоритм гаммирования. Рассмотрим наиболее распространённый вид шифра на основе выражений (1.8) и (1.9), то есть побитового шифра, использующего операцию ИСКЛЮЧАЮЩЕГО ИЛИ (XOR).

Шифр предельно простой, и его суть сводится к переводу сообщения m , и ключа k в бинарный формат посимвольно. Затем вычисляется бинарная последовательность $c = m \oplus k$ с последующим преобразованием ее обратно в символьную последовательность, которая и будет представлять искомый шифротекст. Обратное преобразование осуществляется как $m = c \oplus k$. Прямое преобразование показано на рисунке 2.1. Обратное преобразование можно изобразить аналогично, меняя k и c местами.

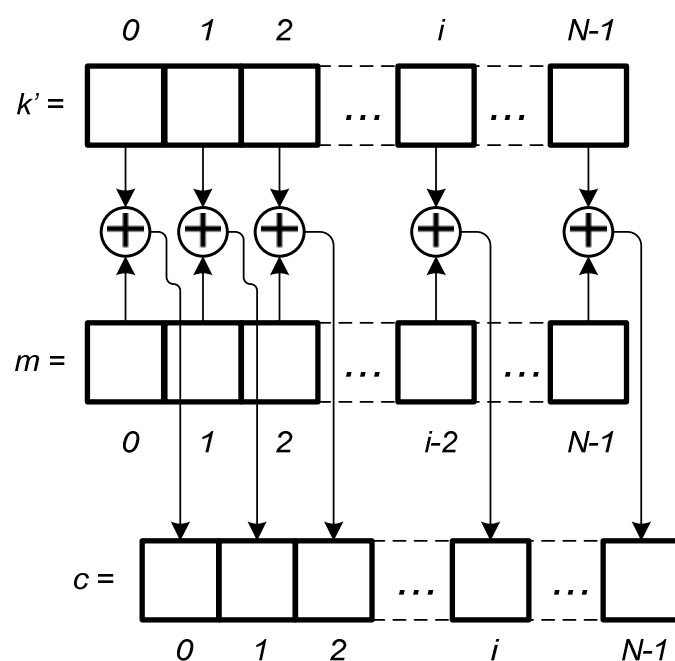


Рис. 2.1. Побитовое XOR-шифрование

Главное достоинство этого вида шифра – бинарное представление как исходного сообщения, так и зашифрованного. То есть его можно применять к изображениям, звуковым, видео-, прочим бинарным файлам.

Если положить, что изначальная длина N передаваемого сообщения неизвестна, то шифр не перестает работать. Необходимо лишь обеспечить последовательность битов ключа k' длины, соответствующей сообщению

m . Эта ключевая последовательность называется *гаммой*. Сам непрерывный шифр именуют *шифром потока*.

Рассмотрим пример работы шифра. При этом шифрование применим к англоязычному посланию. Это сделано постольку, поскольку во всех 8-битных таблицах символов ASCII латинские символы имеют известное положение. К сожалению, для Кириллицы это не верно.

$m = \text{«GIVE \$100 TO LEO»}$, $k = \text{«0123456789ABCDEF»}$.

Ключ k = выбран таким лишь для удобства демонстрации принципа работы кода. Длина ключа соответствует длине сообщения. В реальности приведенным ключом пользоваться не стоит, поскольку он элементарно подбирается по словарям популярных паролей.

Переведем сообщение в коды ASCII:

$m_{\text{ASCII } 10} = \text{«71 73 86 69 32 36 } \underline{\text{49 48 48}} \text{ 32 84 79 32 76 69 79»}$

$m_{\text{ASCII } 16} = \text{«47 49 56 45 20 24 } \underline{\text{31 30 30}} \text{ 20 54 4F 20 4C 45 4F»}$

$m_{\text{ASCII } 2} = \text{«01000111 01001001 01010110 01000101 00100000}$
 $\text{00100100 } \underline{\text{00110001 00110000 00110000}} \text{ 00100000 01010100 01001111}$
 $\text{00100000 01001100 01000101 01001111»}$

Переведем ключ в коды ASCII:

$k_{\text{ASCII } 10} = \text{«48 49 50 51 52 53 54 55 56 57 65 66 67 68 69 70»}$

$k_{\text{ASCII } 16} = \text{«30 31 32 33 34 35 36 37 38 39 41 42 43 44 45 46»}$

$k_{\text{ASCII } 2} = \text{«00110000 00110001 00110010 00110011 00110100}$
 $\text{00110101 00110110 00110111 00111000 00111001 01000001 01000010}$
 $\text{01000011 01000100 01000101 01000110»}$

Вычислим $m_{\text{ASCII } 2} \oplus k_{\text{ASCII } 2}$:

01000111 01001001 01010110 01000101 00100000
 00110000 00110001 00110010 00110011 00110100
01110111 01111000 01100100 01110110 00010100
 00100100 00110001 00110000 00110000 00100000

```

00110101 00110110 00110111 00111000 00111001
00010001 00000111 00000111 00001000 00011001
01010100 01001111 00100000 01001100 01000101
01000001 01000010 01000011 01000100 01000101
00010101 00001101 01100011 00001000 00000000
01001111
01000110
00001001

```

Здесь для наглядности первые строчки – это строчки сообщения $m_{\text{ASCII } 2}$, вторые (написаны курсивом) – строчки ключа $k_{\text{ASCII } 2}$. Третьи строчки (полужирный шрифт) есть $c_{\text{ASCII } 2} = m_{\text{ASCII } 2} \oplus k_{\text{ASCII } 2}$.

Далее переводим $c_{\text{ASCII } 2}$ в шестнадцатеричное представление:

$c_{\text{ASCII } 16} = \text{«77 78 64 76 14 11 07 07 08 19 15 0D 63 08 00 09»},$

$c_{\text{ASCII } 10} = \text{«119 120 100 118 17 07 07 08 25 21 13 99 08 00 09»},$

$c = \text{«w x d v DC1 BEL BEL BS EM NAK CR c BS NUL HT»}.$

В шифре c получилось много служебных элементов таблицы ASCII, которые кодируют не символы, а команды. Это снижает наглядность кода, однако не влияет на его суть и не снижает корректности.

Рассмотрим, как можно атаковать это шифр. Допустим, Мэллори имеет возможность воздействия на шифр, передаваемый от Алисы к Бобу. Однотипные сообщения Алиса передают Бобу периодически. При этом секретный ключ не меняется. Мэллори знает, что Боб должен выплачивать определенную сумму Лео по поручению Алисы.

Допустим, Мэллори хочет узнать, что условия выплат для Лео изменились (например, его повысили в должности). Очевидно, должно измениться и поручение. Раньше поручение выглядело как $m = \text{«GIVE $100 TO LEO»}$, потом стало $m^* = \text{«GIVE $200 TO LEO»}$. Мэллори не знает точного содержания ни m ни m^* , однако она обладает подслушанными шифрами c

и c^* . Разница в m и m^* будет лишь в седьмом символе: $m_6 = \langle 1 \rangle$, $m_6^* = \langle 2 \rangle$. На месте седьмого символа ключа $k_6 = \langle 6 \rangle$, получается в двоичном формате $00110001 \oplus 00110110 = 00000111$, потому $c_6 = \langle \text{BEL} \rangle$. Кроме того, $m_6^* = \langle 2 \rangle$ $m_{6 \text{ ASCII } 10}^* = 50$, $m_{6 \text{ ASCII } 16}^* = 32$, $m_{6 \text{ ASCII } 2}^* = 00110010$. Получается $c_{6 \text{ ASCII } 2}^* = m_{6 \text{ ASCII } 2}^* \oplus k_{6 \text{ ASCII } 2} = 00110010 \oplus 00110110 = 00000100$. Если сложить (при помощи XOR) $c_{6 \text{ ASCII } 2}$ и $c_{6 \text{ ASCII } 2}^*$, то получим $00000111 \oplus 00000100 = 00000111$.

Если сложить шифры c и c^* друг с другом (XOR), то обнулятся все байты кроме тех, которые не совпадают. В нашем случае это c_6 и c_6^* . То есть справедливо:

$$\begin{aligned} c \oplus c^* &= c_0 \oplus c_0^* \parallel c_1 \oplus c_1^* \parallel \dots \parallel c_6 \oplus c_6^* \parallel \dots \\ &\dots \parallel c_{N-1} \oplus c_{N-1}^* = c \oplus c^* = \text{NUL} \parallel \text{NUL} \parallel \dots \\ &\dots \parallel \text{BEL} \parallel \dots \parallel \text{NUL}. \end{aligned} \quad (2.5)$$

Таким образом, можно в автоматическом режиме отслеживать все места, где имеется несовпадение символов в разных шифрах, зашифрованных одним ключом.

Далее, допустим, Мэллори является сообщником Лео и хочет увеличить объем полученных средств путем подмены платежного поручения. Мэллори от Лео становится известно, что в шифровке c от Алибы Бобу находится предписание о выплате \$100. При этом что там содержится еще и в каком порядке ни Мэллори, ни Лео не имеют понятия. После применения вышеописанного алгоритма становится ясно, как выглядит байт, ответственный за сотни в искомой сумме денег.

Для $m = \langle \text{GIVE \$100 TO LEO} \rangle$, нужно получить $m^{**} = \langle \text{GIVE \$900 TO LEO} \rangle$. При этом Мэллори, осуществляющая замену m и m^{**} , не знает структуры m . Все, что известно, это то, что разница в m и m^{**} будет лишь в седьмом символе: $m_6 = \langle 1 \rangle$, $m_6^{**} = \langle 9 \rangle$. Кроме того, у Мэллори есть перехваченный шифр $c = \langle \text{w x d v DC1 BEL BEL BS EM NAK CR c BS NUL} \rangle$.

HT», подлежащий к преобразованию в c^{**} , где денежная сумма увеличивается.

Чтобы осуществить задуманное, Меллори нужно воспользоваться тем фактом, что

$$c^{**} = c \oplus m^{**} \oplus m = k \oplus m \oplus m^{**} \oplus m = k \oplus m^{**}. \quad (2.6)$$

То есть следует взять нужный участок шифра и сложить (XOR) его с поддельным сообщением и сообщением, переданным Алисой. Поскольку задача Мэллори модифицировать только седьмой символ в m , то нужно осуществить преобразование:

$$c_6^{**} = c_6 \oplus m_6^{**} \oplus m_6. \quad (2.7)$$

Значит $c_6 = \text{«BEL»}$, $c_{6 \text{ ASCII } 2} = 00000111$, $m_6^{**} = \text{«9»}$, $m_{6 \text{ ASCII } 2}^{**} = 00111001$, $m_6 = \text{«1»}$, $m_{6 \text{ ASCII } 2} = 00110001$. Собственно, $c_{6 \text{ ASCII } 2}^{**} = 00000111 \oplus 00111001 \oplus 00110001 = 00001111$. То есть $c_{6 \text{ ASCII } 16}^{**} = 0F$, $c_{6 \text{ ASCII } 10}^{**} = 15$, $c_6^{**} = \text{«SI»}$.

В итоге все сообщение, которое подменяет Мэллори будет таким:

$c = \text{«w x d v DC1 BEL SI BS EM NAK CR c BS NUL HT»}$.

Отдельно следует отметить, что если злоумышленник получит в свое распоряжение одновременно исходное сообщение c и его шифр k , то он элементарно восстановит ключ k . Действительно, $k = c \oplus m$. Потому использование одинакового ключа дважды серьезно подрывает надежность шифрования и этого следует всячески избегать.

2.2. Простейшие шифры перестановки

Помимо **шифров подстановки**, принято выделять **шифры перестановки** при этом выделяют простую перестановку и перестановку по ключу.

Простая перестановка. Выбирается ключ $k \in \mathbb{N}$. Далее исходный текст m записывается в матрицу по строкам. Матрица имеет k столбцов. Число строк определяется условием исчерпания текста m , то есть равно $\lceil |m|/k \rceil$.

Пример. m = «ПРИШЕЛ УВИДЕЛ ПОБЕДИЛ», k = 8.

П	Р	И	Ш	Е	Л		У
В	И	Д	Е	Л		П	О
Б	Е	Д	И	Л			

Шифр c = «ПВБРИЕИДДШЕИЕЛЛ__П_УО_»

Стойкость этого шифра, очевидно, невелика и связана с прямым перебором множества возможных ключей, представляющих натуральные числа, ограниченные мощностью исходного текста.

Перестановка по ключу. Здесь в качестве ключа выбирается некоторое слово k , длиной $|k|$. Далее осуществляется запись текста m в матрицу размером $|k| \times [|m|/|k|]$ аналогично методу простой перестановки. После этого осуществляется сортировка столбцов матрицы согласно алфавитному порядку символов ключа k . Шифрованное сообщение c получается путем извлечения символов из матрицы по столбцам.

Пример. m = «ПРИШЕЛ УВИДЕЛ ПОБЕДИЛ». k = «ЮЛИАН».

Ю	Л	И	А	Н
П	Р	И	Ш	Е
Л		У	В	И
Д	Е	Л		П
О	Б	Е	Д	И
Л				

А	И	Л	Н	Ю
Ш	И	Р	Е	П
В	У		И	Л
	Л	Е	П	Д
Д	Е	Б	И	О
				Л

Шифр c = «ШВ_Д_ИУЛЕ_Р_ЕБ_ЕИПИ_ПЛДОЛ»

Здесь прямая стойкость уже определяется числом возможных перестановок для выбранной длины ключа то есть $N^{|k|}|k|!$. но поскольку и сама длина ключа неизвестна, то сложность будет порядка $N^{|k|}(|k| + 1)!$. При больших $|k|$ будем получать шифр, практически не взламываемый совре-

менными средствами. Однако, учитывая повтор по строкам и знания алфавита языка, на котором построен шифр, можем достаточно просто произвести успешный криптоанализ. Для этого можно разбирать шифр c на части, постепенно увеличивая размер блока покуда не будет достигнуто значение $|k|$. Каждую из частей анализируем на предмет того, какие слова можно составить из элементов блока. Естественно, для этого нужно иметь словарь слов и словосочетаний соответствующего языка.

Матричный шифр. В качестве ключа выступают два слова k_1 и k_2 , длиной, соответственно, n_1 и n_2 . Далее формируется матрицы размерами $n_1 \times n_2$. В ячейки матриц записывается текст m , подлежащий шифровке. Число матриц выбирается так, чтобы в них поместилось всё сообщение. Строки матриц упорядочиваем согласно алфавитному порядку символов k_1 , а потом аналогично столбцы согласно k_2 . Шифр c получается путем выписывания полученных символов по столбцам.

Рассмотрим пример шифра:

$m = \text{«ПРИШЕЛ УВИДЕЛ ПОБЕДИЛ»}$, $k_1 = \text{ТАНЯ}$; $k_2 = \text{ИРА}$

	И	Р	А		И	Р	А		А	И	Р		А	И	Р
Т	П	Р	И	Т	Л		П	А	Л	Ш	Е	А	Е	О	Б
А	Ш	Е	Л	А	О	Б	Е	Н	В		У	Н	Л	Д	И
Н		У	В	Н	Д	И	Л	Т	И	П	Р	Т	П	Л	
Я	И	Д	Е	Я				Я	Е	И	Д	Я			

Полученный шифр: $c = \text{«ЛВИЕШ ПИЕУРДЕЛП ОДЛ БИ »}$.

Мощность множества ключей определяется суммарной длиной k_1 и k_2 и перестановками на их символах. То есть $|K| = N^{|k_1|} \cdot N^{|k_2|} \cdot |k_1|! \cdot |k_2|!$, что весьма огромным числом является при больших длинах ключа. Однако здесь используются блоки, ключ для которых, периодически повторяется. Кроме того, сами символы никак не маскируются, что позволяет

осуществлять взлом шифра путем анализа блоков по словарю (выбором тех слов и их сочетаний, которые можно по нему составить).

Шифр матричной перестановки с гаммированием (ADFGX). Этот вид шифра использует для кодирования каждого символа исходного текста пару символов или иных знаков, получаемых из ключевой таблицы, в которой перетасован базовый алгоритм. Поскольку в английском алфавите 26 букв, то пару из них объединяют в одну и получают матрицу 5×5 , строки и столбцы которой маркируют как ADFGX. Для русского алфавита лучше использовать матрицу 6×6 , а индексацию осуществлять просто числами.

Например, зашифруем

k_T – таблица:

	1	2	3	4	5	6
1	Ф	У	–	Г	Ю	Ъ
2	Ь	А	В	К	Д	Э
3	!	Ч	Б	Л	Е	М
4	Х	Й	Ы	Ё	Т	Н
5	Ц	Ш	Ж	И	О	С
6	Щ	,	З	Я	П	Р

m = «ПРИШЕЛ УВИДЕЛ ПОБЕДИЛ»,

c_T = «646642523534131223422544341365553335255434» (64 66 42 52 35 34 13 12 23 42 25 44 34 13 65 55 33 35 25 54 34).

Здесь применяется по сути перестановка, просто иначе закодированная. Криптостойкость к частотному анализу на этом этапе неудовлетворительная.

Это был первый этап шифровки. Второй этап состоит в упрощенном варианте перестановки по ключу. Для этого берется второй ключ k_P в виде

слова и сообщение из чисел записывается под ним по строкам. Потом происходит сортировка ключевого слова по алфавиту и вместе с ним сортируются соответствующие столбцы. Далее по столбцам выписывают числа, которые и составляют конечный шифр s .

В качестве второго ключа возьмем $k_p = \text{«ЮЛИЙЦЕЗАРЬ»}$.

Ю	Л	И	Й	Ц	Е	З	А	Р	Ь
6	4	6	6	4	2	5	2	3	5
3	4	1	3	1	2	2	3	4	2
2	5	4	4	3	4	1	3	6	5
5	5	3	3	3	5	2	5	5	4
3	4								

А	Е	З	И	Й	Л	Р	Ц	Ю
2	2	5	6	6	4	3	4	6
3	2	2	1	3	4	4	1	3
3	4	1	4	4	5	6	3	2
5	5	2	3	3	5	5	3	5
					4			3

Значит, шифр будет

$m_p = \text{«23355545521261436343445543465413363253»}$

Стойкость этого шифра достаточно высокая, но при этом все равно есть возможность применить методы частотного анализа с перебором.

3. Блочные и поточные шифры

3.1 Блочные шифры

Для достаточно больших сообщений приходится применять один и тот же ключ. В простейших шифрах подстановки этот шифр применяется к отдельным символам или их небольшим группам. Однако, для шифров перестановки получаются уже достаточно большие группы символов, над которыми осуществляются однотипные действия. Самое главное свойство при этом – независимость блоков друг от друга. Это позволяет извлекать независимые блоки из сообщения без дешифровки его полностью. Кроме того, процесс шифровки и дешифровки идеально распараллеливается, что весьма актуально в наше время многопроцессорных и распределенных вычислительных систем.

Естественно, что идеальным шифром будет такой, при котором размер блока совпадает с размером исходного сообщения. Однако, в этом случае размер ключа должен быть равен длине исходного сообщения. Кроме того, процесс перестановки весьма затратная во вычислительным мощностям операция.

Для того, чтобы доказать эффективность использования блочных шифров, следует провести следующие действия. Нужно осуществить преобразование данных, осуществляющее подстановку SC (substitution cipher) в последовательность исходного сообщения, которая, по своей сути будет представлять собой кодирование информации и представление ее в новой форме. Затем вводим случайные перестановки, возможно в сочетании со случайными функциями TC (transposition cipher).

Следует доказать, что применение композиции $E(m, k_{SC}, k_{TC}) = TC(SC(m, k_{SC}), k_{TC})$ на всех ключах k_{SC} и k_{TC} приводит к практической невозможности вскрытия шифра противником. Функция E биективна для

любых комбинаций k_{SC} и k_{TC} . Доступных перестановок и способов кодирования чрезвычайно много. Для того, чтобы иметь возможность получать из базовой ключевой последовательности небольшой длины последовательность гораздо большую по размеру применяют псевдослучайные перестановки (pseudo random permutation, PRP) и/или функции (PRF), такие что ни один эффективный противник не может отличить их от настоящих. Далее нужно доказать, что с полученным шифром эффективный противник не справится. Если шифр на основе комбинации $TC(SC(m, k_{SC}), k_{TC})$ при использовании PRP и/или PRF действительно не подвержен взлому, то он выбирается в качестве рабочего.

Принцип устройства блочного шифрования показан на рисунке 3.1.

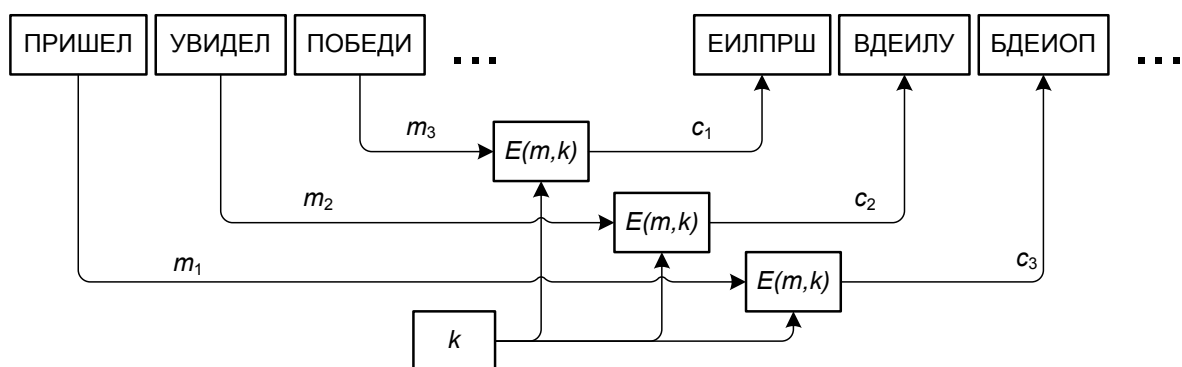


Рис. 3.1. Принцип работы блочного шифра

На практике блоки исходного текста шифруются индивидуально, но с использованием ключей потока для шифрования всех сообщений блок за блоком. Шифр будет блочным, если применяется каждому блоку, но на уровне всего сообщения, его можно считать поточным, просто взаимовлияния блоков нет. Каждый блок может использовать различный ключ, который был сгенерирован заранее или в течение процесса шифрования.

Недостатком применения блочного шифра является сохранение взаимного расположения блоков, что может быть использовано при вскрытии. Наглядный пример такой особенности, взятый из популярных открытых источников, показан на рисунке 3.2.



Рис. 3.2. Пример сохранения низкочастотной составляющей в сообщении

3.2. Поточные шифры

В шифрах потока в один момент времени осуществляется шифрование над одним символом или их группой. При этом последующий блок может зависеть от предыдущего. Для шифрования используется псевдослучайная функция PRF , которая генерирует набор ключей, получаемый из некоторого базового ключа k . К сожалению, у любой $PRF(k)$ для порождаемой последовательности k^* спустя конечное число циклов применения этой PRF будет наблюдаться повтор k^* . Это значит, что применяемую связку PRF и k следует брать таким образом, чтобы противник не смог достигнуть этого периода.

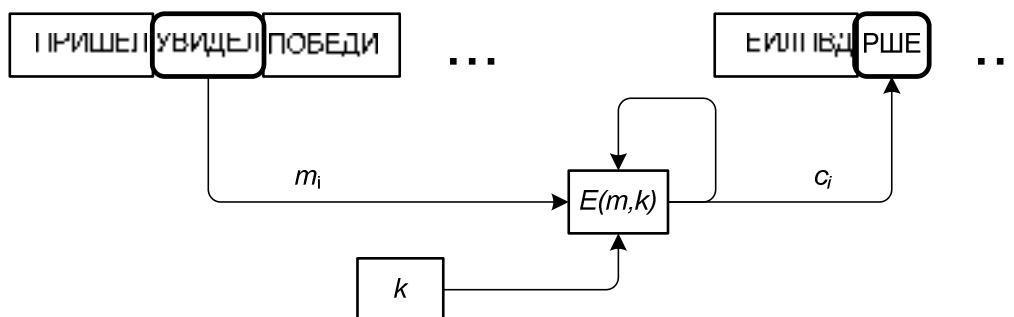


Рис. 3.3. Принцип работы поточного шифра

Недостатком применения поточного шифра является тот факт, что для расшифровки части сообщения, находящейся достаточно далеко от

начала сообщения требуется произвести операцию расшифровки всего предшествующего нужному блоку потока.

Общая схема функционирования такого шифра показана на рисунке 3.3. Символы текста принимаются алгоритмом шифрования по одиночке или блоком. Символы зашифрованного текста также создаются по одному или блоками в один и тот же момент времени. Значения могут зависеть от исходного текста или символов зашифрованного текста. Значения могут также зависеть от предыдущих ключевых значений.

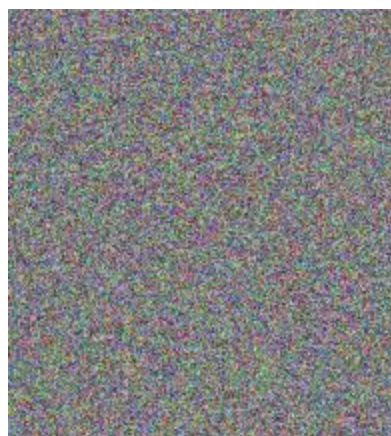


Рис. 3.4. Пример скрытия низкочастотной составляющей в сообщении

Правильно сформированный шифр на основе хороших *PRF* производит в достаточной мере статистически однородный поток на выходе. Если период выхода $PRF(k)$ больше чем длина зашифрованного сообщения m , то такой шифр справляется с задачей устранения частотных особенностей в шифруемом сообщении. Пример, показывающий принципиальную суть описываемого показан на рисунке 3.4.

3.3. Режимы комбинации шифрования блоков

Самым простым методом работы шифра является режим так называемой *электронной кодовой книги* (Electronic Codebook – ECB). По своей сути это режим блочной шифрации без связи между блоками. Схема EBC, работающего в прямом направлении показана на рисунке 3.5.

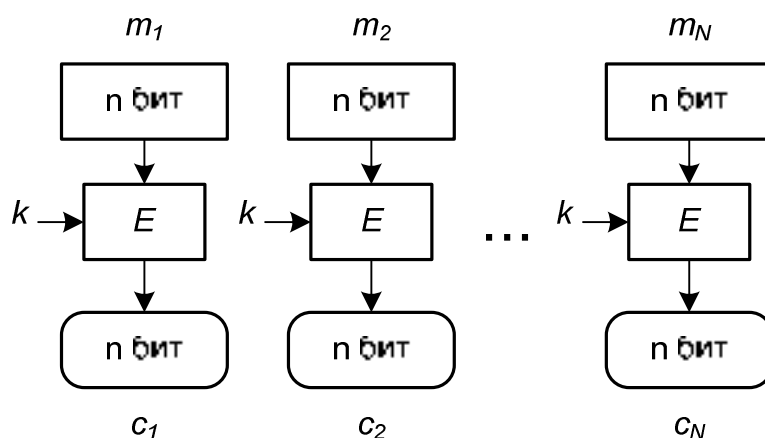


Рис. 3.5. Шифрование в режиме ECB

Расшифровка представлена на рисунке 3.6.

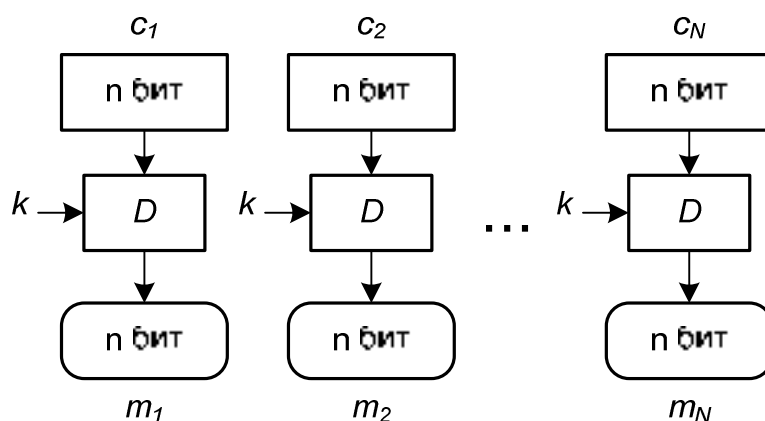


Рис. 3.6. Дешифрование в режиме ECB

Здесь для шифрования применяется следующее выражение:

$$c_i = E_k(m_i). \quad (3.1)$$

Расшифровка такова:

$$m_i = D_k(c_i). \quad (3.2)$$

Преимущества режима ECB: независимость шифрования / дешифрования блока от остальных блоков. Возможность параллельной обработки блоков.

Недостатки ECB: при использовании одного ключа не скрываются низкочастотные характеристики сообщения с периодом более чем $|m_i|$.

При атаке «человек посередине», реализуемой ролью Мэллори возможно извлечение отдельных блоков из сообщения или их дублирование. Возможна подмена блоков при условии наличия известного ключа. При этом Алиса и Боб не смогут идентифицировать факт атаки без вспомогательных средств.

Этот режим шифрования может быть рекомендован для шифрования на локальных хранилищах данных для предотвращения несанкционированного доступа.

Режим сцепления блоков шифрованного текста (CBC – Cipher Block Chaining). В этом режиме блоки соединятся в поток, в котором каждый последующий блок зависит от предыдущего. Здесь используется операция ИСКЛЮЧАЮЩЕГО ИЛИ, побитово примененная к исходному сообщению и шифру, полученному на предыдущем раунде преобразования m_i в c_i . Для возможности старта процесса шифрования на вход первому блоку подается поток бит, называемый инициализационным вектором IV .

Аналитическая запись шифрования:

$$\begin{cases} c_0 = IV \\ c_i = E_k(m_i \oplus c_{i-1}). \end{cases} \quad (3.3)$$

Аналитическая запись дешифровки:

$$\begin{cases} c_0 = IV \\ m_i = c_{i-1} \oplus D_k(c_i). \end{cases} \quad (3.4)$$

Режим CBC не используется при шифровании и дешифровании массивов файлов с произвольным доступом, потому что шифрование и дешифрование требуют доступа к предыдущим массивам. Режим CBC применяется для установления подлинности сообщения.

Достоинства CBC: режим CBC режимом устраняет ECB. Блоки открытого текста маскируются блоками шифротекста, полученными от предыдущего блока.

Если произойдет ошибка в одном бите шифротекста, то она распространится и на следующий блок. Но на последующие блоки (через один) ошибка не распространится, поэтому режим CBC также называют самовосстанавливающимся.

Процедура дешифровки легко распараллеливается.

Недостатки CBC: процедура шифрования трудно распараллеливается.

Если происходит потеря или вставка битов, то шифр разрушается. Для устранения этого недостатка нужно применять механизм выравнивания блоков.

Проблемы безопасности CBC: Ева может видеть, что одинаковые блоки сообщения m зашифровываются в различные блоки зашифрованного текста c . Если два сообщения равны, то шифры совпадают, при условии одинаковых IV . Появляется возможность определения начала изменения данных при изменении шифротекста.

Для очень крупных сообщений (32 Гбайта при длине блока 64 бита) возможно применение атак, основанных на структурных особенностях открытого текста (*парадокс дней рождений*).

Мэллори может пристыковать свои блоки зашифрованного текста в конец потока зашифрованного текста.

Мэллори имеет возможность переделать открытый текст при перемещении блоков (но без ключа получается бессмысленный набор символов).

Прямой процесс шифрования CBC показан на рисунке 3.7.

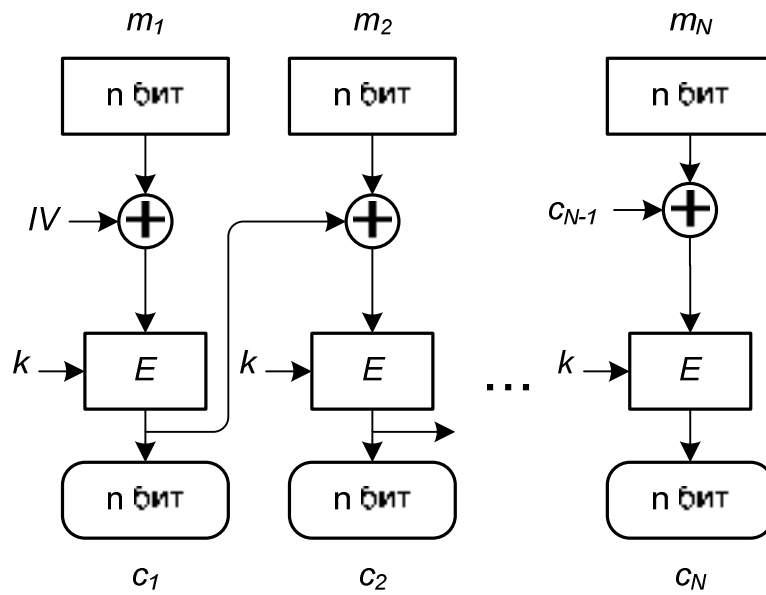


Рис. 3.7. Шифрование в режиме CBC

Процесс обратного преобразования показан на рисунке 3.8.

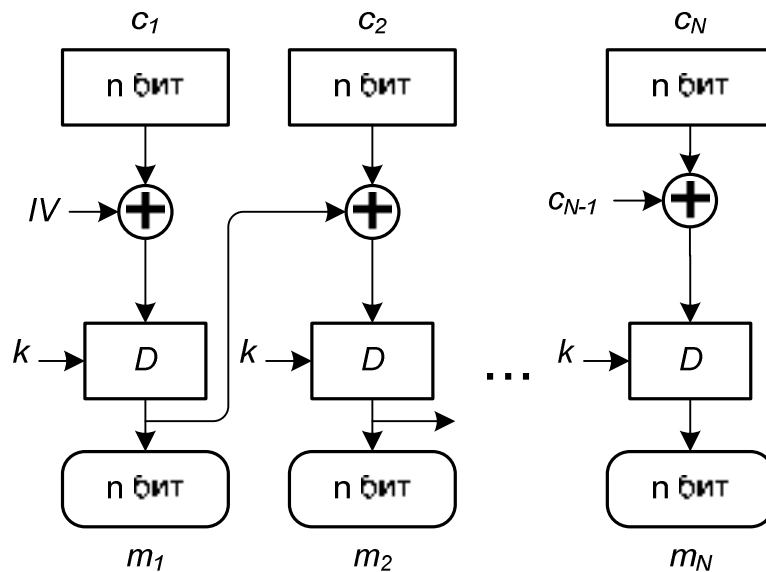


Рис. 3.8. Дешифрование в режиме CBC

Следующим видом связки блоков в поток является **режим кодированной обратной связи** (Cipher Feedback Mode CFB).

Здесь аналогично CBC для возможности старта процесса на вход первому блоку подается поток бит IV .

Процесс шифрования:

$$\begin{cases} c_0 = IV \\ c_i = m_i \oplus E_k(c_{i-1}). \end{cases} \quad (3.5)$$

Дешифровка:

$$\begin{cases} c_0 = IV \\ m_i = c_i \oplus E_k(c_{i-1}). \end{cases} \quad (3.6)$$

Преимущества: существует более сложный вариант использования режима, когда размер блока CFB не совпадет с размером блока шифра. В этом режиме не требуется дополнение блоков, поскольку размер блоков выбирается так, чтобы удовлетворить размеру блока данных, который нужно зашифровать (например, символ).

Система не должна ждать получения большого блока данных для начала шифрования. Процесс шифрования выполняется для маленького блока данных (таких как символ).

Ошибка в шифротексте делает невозможным расшифровку блока в котором она произошла и следующего за ним, однако не распространяется на последующие блоки.

Фиксированные блоки открытого текста маскируются блоками шифротекста.

Недостатки CFB: менее эффективен, чем CBC или ECB, поскольку маленький блок шифруется основным блочным шифром.

Если в режиме CFB с полноблочной обратной связью имеется два идентичных блока шифротекста, то результат шифрования на следующем шаге будет тем же.

Одинаковые блоки исходного текста, принадлежащие одному и тому же самому сообщению, зашифровываются в различные блоки.

Одним и тем же ключом может быть зашифровано больше чем одно сообщение, но значение IV должно быть изменено для каждого сообщения.

Мэллори может добавить некоторый блок зашифрованного текста к концу потока зашифрованного текста.

Процесс шифрования плохо распараллеливается.

Режим работы CFB может использоваться, чтобы зашифровать блоки небольшого размера, такие как один символ или бит. Нет необходимости в дополнении, потому что размер блока исходного текста обычно устанавливается (8 для символа или 1 для бита).

Прямой процесс шифрования представлен на рисунке 3.9.

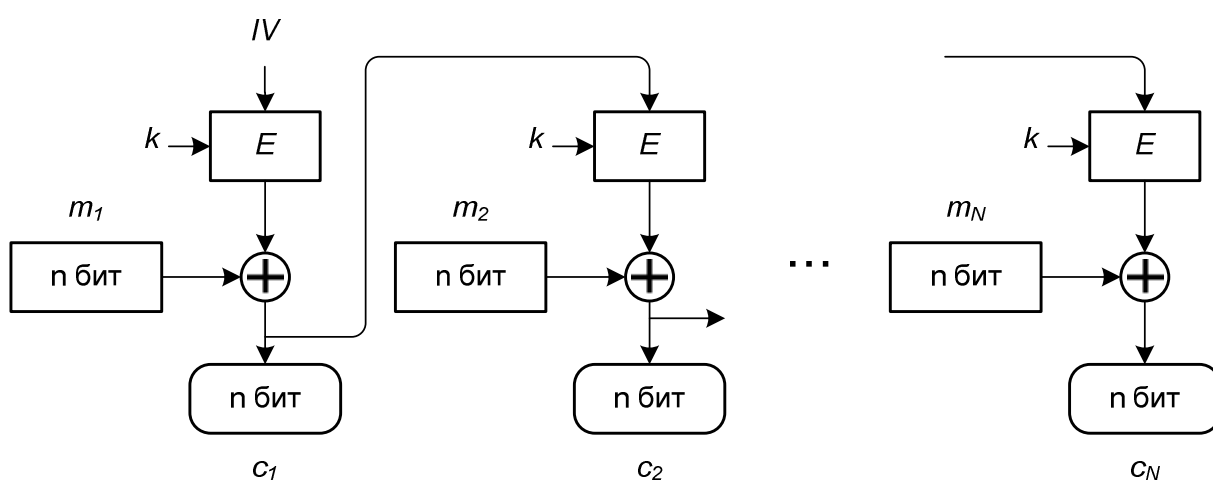


Рис. 3.9. Шифрование в режиме CFB

Процесс обратного преобразования показан на рисунке 3.10.

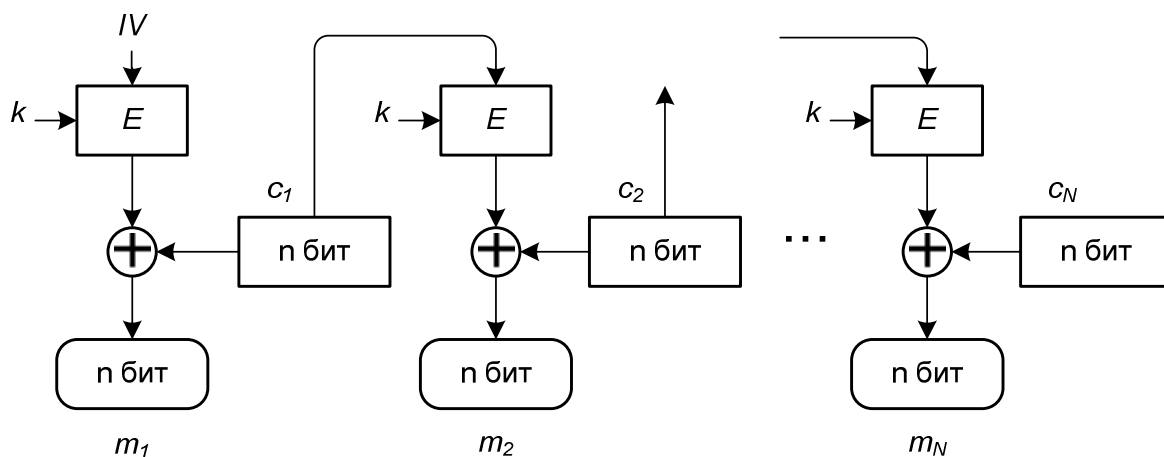


Рис. 3.10. Дешифрование в режиме CFB

Режим внешней обратной связи (Output Feedback – OFB). Режим называют также «режим обратной связи по выходу». Режим OFB похож на CFB, но каждый бит в зашифрованном тексте независим от предыдущего бита или битов. Это позволяет избежать распространения ошибок. Если при передаче возникает ошибка, она не затрагивает следующие биты. Режим OFB превращает блочный шифр в синхронный шифр потока. Здесь генерируются ключевые блоки, являющиеся результатом сложения с блоками открытого текста. Зеркальное отражение в зашифрованном тексте производит зеркально отраженный бит в открытом тексте в том же самом местоположении. Это дает возможность многим кодам с исправлением ошибок функционировать как обычно, даже когда исправление ошибок применено перед кодированием. Для возможности старта процесса на вход первому блоку подается поток бит инициализации IV .

Процесс шифрования:

$$\begin{cases} \theta_0 = IV \\ \theta_i = E_k(\theta_{i-1}). \\ c_i = m_i \oplus \theta_i \end{cases} \quad (3.7)$$

Дешифровка:

$$\begin{cases} \theta_0 = IV \\ \theta_i = E_k(\theta_{i-1}). \\ m_i = c_i \oplus \theta_i \end{cases} \quad (3.8)$$

Особенности: каждая операция OFB зависит от всех предыдущих и не может быть выполнена параллельно. Но открытый текст или зашифрованный текст используются только для конечного сложения, потому что операции блочного шифра могут быть выполнены заранее, позволяя выполнять заключительное шифрование параллельно с открытым текстом.

Ошибки, возникающие в результате передачи при дешифровании, локализуются в пределах одного блока.

Для OFB ключ потока не зависит от исходного текста или зашифрованного текста.

Проблемы безопасности: такие же, как и в режиме CFB. Одинаковые блоки исходного текста, принадлежащие тому же самому сообщению, зашифровываются в различные блоки.

Изменение в зашифрованном тексте затрагивает исходный текст, зашифрованный в приемнике.

Прямой процесс OFB показан на рисунке 3.11.

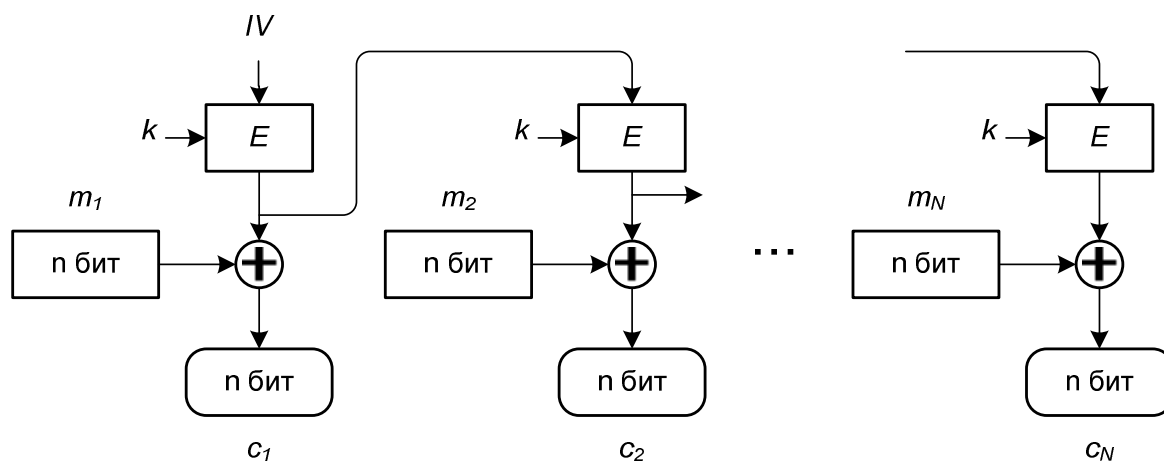


Рис. 3.11. Шифрование в режиме OFB

Процесс обратного преобразования OFB показан на рисунке 3.12.

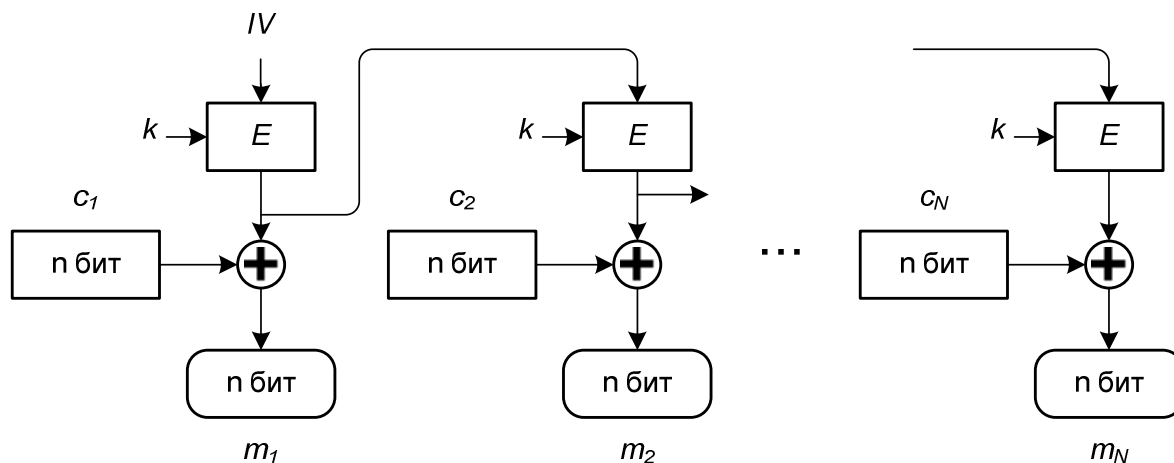


Рис. 3.12. Дешифрование в режиме OFB

Режим счетчика (Counter – CTR). В этом режиме нет обратной связи. Псевдослучайный поток ключей достигается с помощью счетчика. Счетчик на n бит инициализируется значением IV , определенным заранее, и увеличивается по заранее определенному правилу ($k_i = k_{i-1} + 1 \bmod 2^K$). Для обеспечения случайности величина приращения может зависеть от номера блока. Исходный текст и блок зашифрованного текста имеют один и тот же размер блока, как и основной шифр.

Процесс шифрования:

$$c_i = m_i \oplus E_k(Ctr_i), i = 1, 2, \dots, K \quad (3.9)$$

Дешифровка:

$$m_i = c_i \oplus E_k(Ctr_i), i = 1, 2, \dots, K \quad (3.10)$$

Здесь Ctr_i – значение счетчика для i -го блока. Этот счетчик соединяется с некоторым случайным числом IV . Просто к нему приписывается значение счетчика (конкатенируется), то есть $IV \parallel Ctr_i$.

Особенности: при отсутствии обратной связи алгоритмы шифрования и расшифровки в режиме CTR могут выполняться параллельно. Более того, большие объёмы вычислений, связанные с шифрованием значений счетчика могут быть выполнены заранее, до того как открытый текст или шифротекст окажутся доступными.

Безопасность: проблемы безопасности для режима CTR те же самые, что и для режима OFB.

Прямой процесс режима CTR показан на рисунке 3.13.

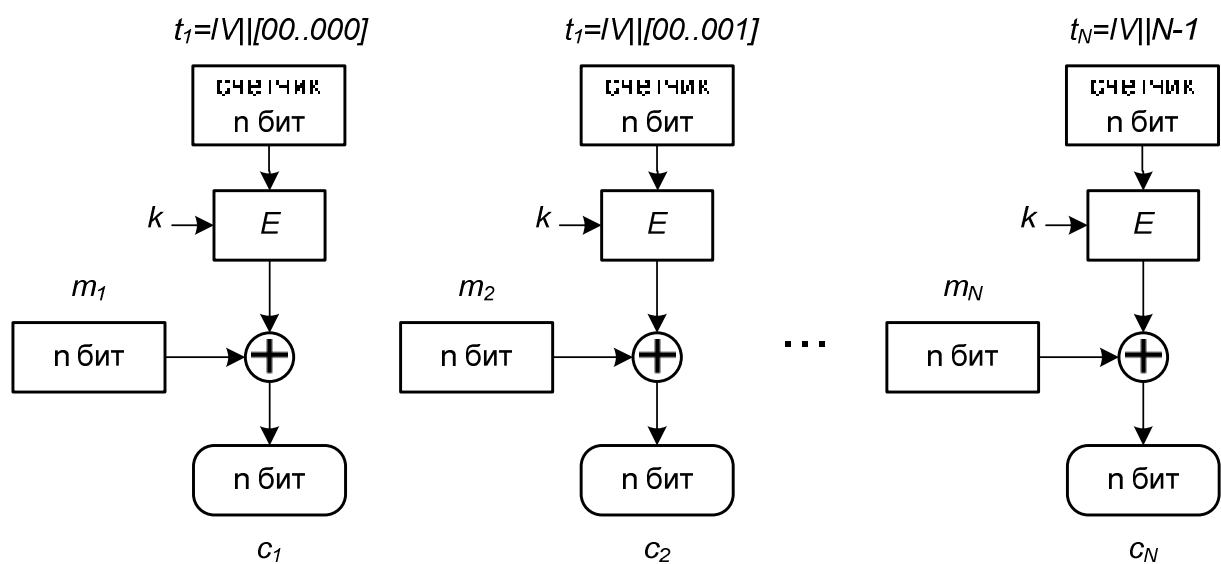


Рис. 3.13. Шифрование в режиме CTR

Процесс обратного преобразования показан на рисунке 3.14.

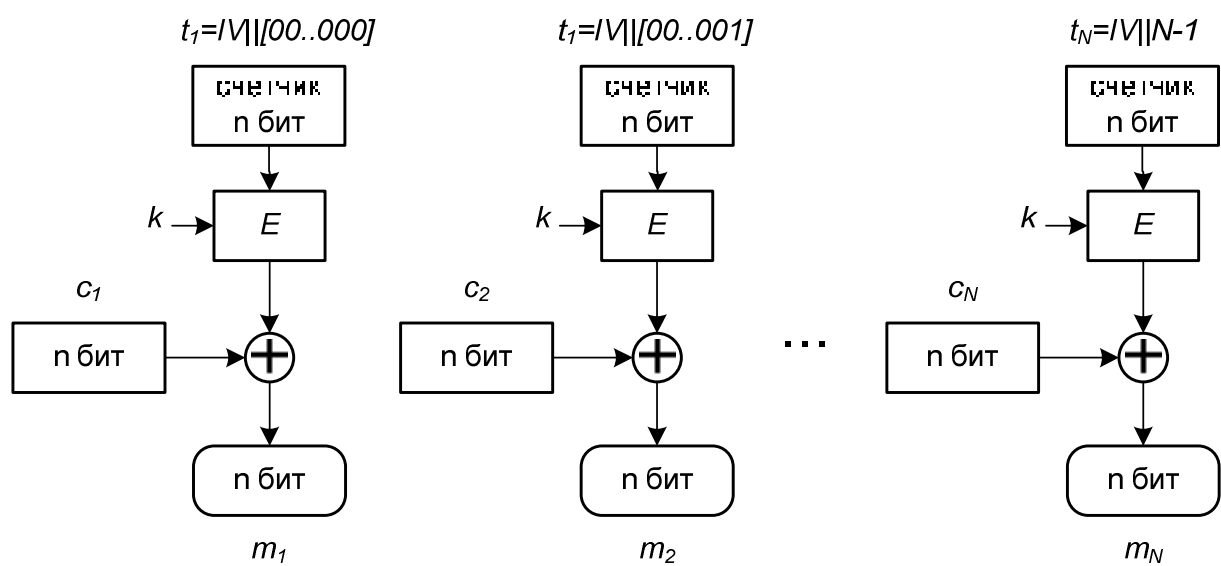


Рис. 3.14. Дешифрование в режиме CTR

4. Современные симметричные шифры потока

Несмотря на наличие методов и средств для возможности обмена сообщениями в режиме, не требующем общего секретного ключа, симметричная криптография сегодня является актуальной. Главная причина этой актуальности в возможности шифровки огромных объемов информации относительно небольшими вычислительными мощностями. К сожалению, для асимметричной криптографии сложность шифрования нелинейно зависит от объема передаваемых данных. Потому в современных криптографических системах применяются комбинированные протоколы информационного обмена, в которые входят элементы как симметричной, так и асимметричной криптографии.

Рассмотрим далее несколько популярных методов шифрования, принимаемых на уровне отраслевых или государственных стандартов. Рассмотренные ранее режимы шифрования по комбинации блоков в поток допускают использование блочных шифров для шифрования сообщений или файлов в больших модулях (ECB, CBC и CTR) и маленьких модулях (OFB и OFB), иногда необходимо передать поток для того, чтобы зашифровать маленькие единицы информации – символы или биты. Шифры потока более эффективны для обработки в реальном масштабе времени. В качестве шифров потока рассмотрим RC4, A5 и A6.

4.1. Алгоритм RC4

RC4 –поточковый шифр с переменным размером ключа, разработанный в 1987 году компанией RSA Data Security. Изначально он был полностью конфиденциальным и для его использования требовалась подпись о неразглашении. Однако, вскоре возникла анонимная публикация, вскрывающая особенности его функционирования. Алгоритм применяется во

многих системах (например, в протоколе SSL/TLS, для шифрования паролей в Windows NT, IEEE 802.11 (беспроводный стандарт LAN) и др.)

Алгоритм работает в режиме OFB, то есть гамма не зависит от открытого текста. Используется так называемый, s -блок размером 8×8 : $s_0, s_1, \dots, s_{255}$. Элементы представляют собой перестановку чисел от 0 до 255. Сама перестановка зависит от ключа переменной длины. В алгоритме применяются два счетчика i и j с нулевыми начальными значениями.

Для генерации очередного псевдослучайного байта необходимо выполнить последовательность преобразований.

$$\begin{cases} i = (i + 1) \bmod 256, \\ j = (j + s_i) \bmod 256. \end{cases} \quad (4.1)$$

Далее меняются i -й и j -й байты местами:

$$s_i \leftrightarrow s_j. \quad (4.2)$$

Вычисляется новый индекс t :

$$t = (s_i + s_j) \bmod 256. \quad (4.3)$$

Наконец, получают значение очередного байта ключа k' :

$$k' = s_t. \quad (4.4)$$

Полученный байт используется в операции XOR с открытым текстом m для получения шифротекста. Аналогичные действия применяются для расшифровки. Таким образом, формируется ключевая гамма для соответствующего алгоритма, суть которого показана на рисунке 2.1.

Начальное значение вектора s инициализируется следующим алгоритмом. Сначала происходит его линейное заполнение числами от 0 до 255 с шагом 1:

$$s_0 = 0, s_1 = 1, \dots, s_{255} = 255. \quad (4.5)$$

Затем инициализируют значением ключа другой 256-байтовый массив $(k_1, k_2, \dots, k_{255})$. Если ключ оказался короче, чем надо, он повторяется нужное число раз. После этого применяют алгоритм:

Индекс j делают равным «0»:

$$\begin{aligned} &\text{для } i = 0, 1, \dots, 255: \\ &\begin{cases} j = (j + s_i + k_i) \bmod 256 \\ s_i \leftrightarrow s_j \end{cases} \end{aligned} \quad (4.6)$$

Схема работы алгоритма перестановок для генерации ключевой гаммы k_* показана на рисунке 4.1.

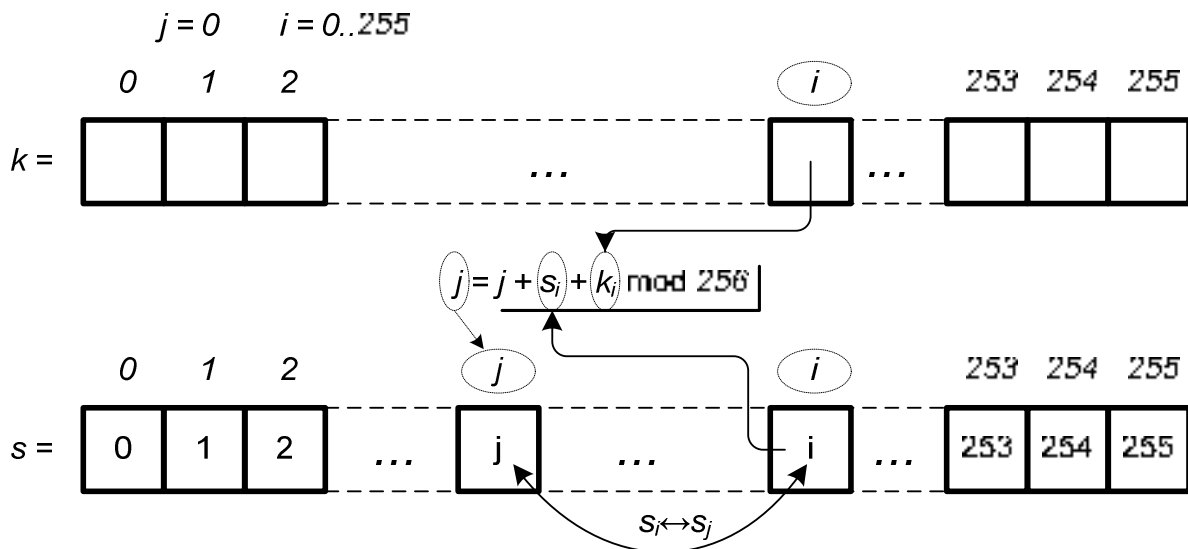


Рис. 4.1. Инициализация вектора s в алгоритме RC4

Считается, что этот алгоритм устойчив к дифференциальному и линейному методам криптоанализа и не содержит коротких циклов. (Опубликованных криптоаналитических результатов нет. Алгоритм RC4 может находиться примерно в 21700 (т. е. $256! \cdot 256^2$) возможных состояниях. При использовании s -блок медленно изменяется: i гарантирует изменение каждого элемента, а j – случайный характер этих изменений. Алгоритм чрезвычайно прост и доступен к воспроизведению по памяти. Помимо высокой устойчивости к криптоанализу, этот алгоритм очень быстр и может использоваться для генерации ключевой последовательности при потоковом шифровании.

Алгоритм RC4 можно обобщить на s -блоки и слова больших размеров. Выше была описана 8-битовая версия RC4. Можно определить

16-битовый алгоритм RC4 с s -блоком размером $16 \cdot 16$ (100 Кбайт памяти) и 16-битовым словом. Начальная установка займет больше времени, поскольку нужно заполнить 65536-элементный массив. Однако, получившийся алгоритм должен работать быстрее.

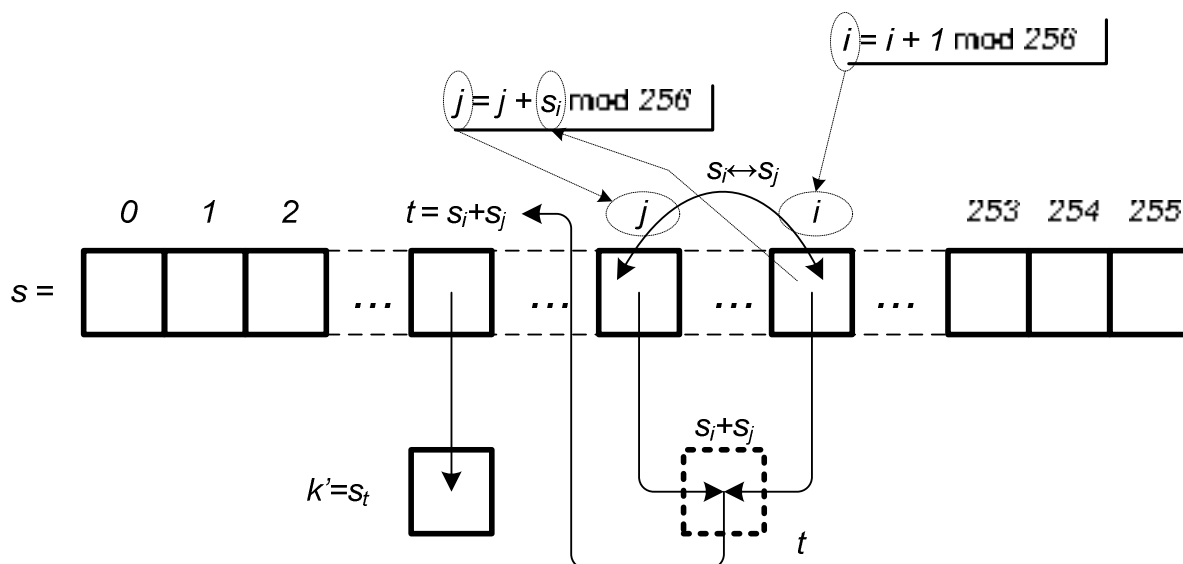


Рис. 4.2. Генерация псевдослучайного байта гаммы алгоритме RC4

Алгоритм RC4 больше не является коммерческим секретом, однако формально, все еще в собственности компании RSA Data Security. Потому в системах, не имеющих лицензионной связи с вышеупомянутой фирмой, данный алгоритм носит название ARCFOUR или ARC4 (англ. Alleged RC4).

4.2. Алгоритмы A5 как база шифрования в сетях GSM

Далее рассмотрим практическое применение алгоритмов потокового шифрования на примере протоколов обеспечения безопасности в сетях сотовой связи стандарта GSM. На самом деле, стандарты безопасности в GSM представляют собой целый набор соответствующих методов и средств. Официально эти алгоритмы не раскрыты до сих пор и формально являются объектами под грифом «для служебного пользования». Однако, ввиду большой распространенности GSM, алгоритмы широко известны.

В основе безопасности GSM лежат три секретных алгоритма:

A3 – алгоритм аутентификации, защищающий телефон от клонирования;

A8 – алгоритм генерации криптоключа (однонаправленная функция, берущая фрагмент выхода от A3, превращая его в сеансовый ключ для A5);

A5 – алгоритм шифрования звукового потока.

Собственно, остановимся на рассмотрении A5. Видов шифрования A5 выделяют несколько:

A5/0 – режим работы алгоритма шифрования без шифрования;

A5/1 – наиболее распространенный на сегодня поточный шифр.

A5/2 – сильно ослабленная версия A5/1, изначально предназначавшаяся для экспорта (в настоящее время не используется);

A5/3 – блочный шифр, разработанный в 2002 году с целью замены A5/1. Сегодня получил распространение в 3GPP сетях. У алгоритма найден ряд уязвимостей, но практических атак пока не реализовано.

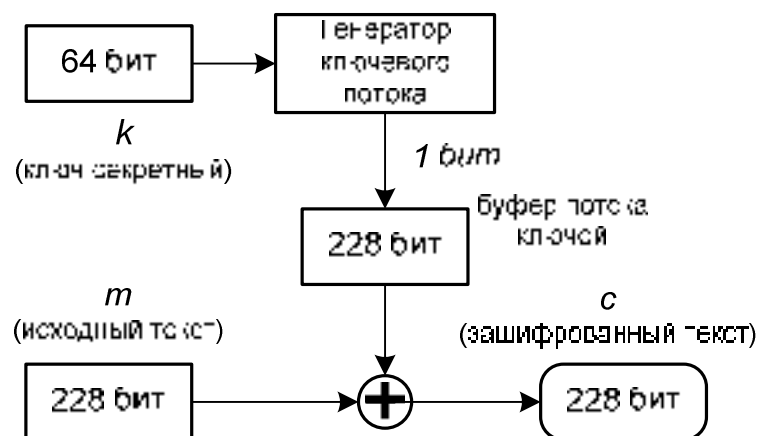


Рис. 4.3. Общий принцип работы A5/1

Итак, рассмотрим A5/1 как наиболее активный реально используемый шифр. Телефонная связь в GSM осуществляется как последовательность кадров на 228 битов, при этом каждый кадр длится 4,6 миллисекунды. A5/1 создает поток бит исходя из ключа на 64 бита. Разрядные потоки

собраны в буфере по 228 бит, чтобы складывать их по модулю два с кадром на 228 битов. В общем A5/1 является стандартным потоковым шифром, общая схема которого показана на рисунке 4.3.

Генератор ключевого потока использует три линейных регистра сдвига с обратной связью (LFSR) на 19, 22, 23 бита (рисунок 4.4). Регистры содержат биты символов и синхронизации. Всего получается 64 бита.

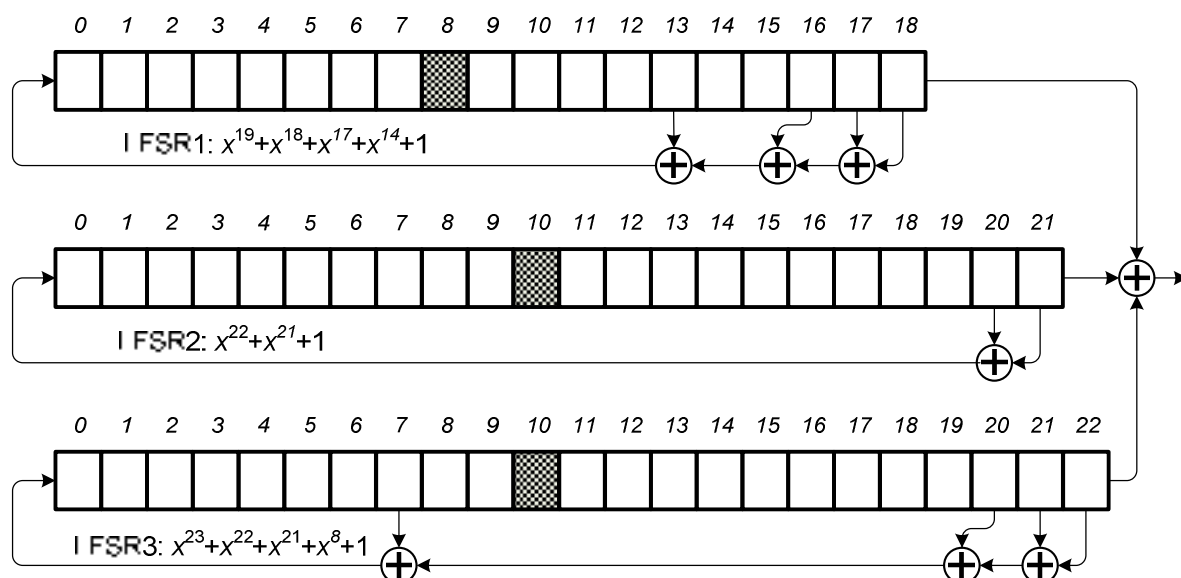


Рис. 4.4. Генератор ключевого потока

LFSR состоит из последовательности бит определенной длины (регистр) и обратной связи. На каждой итерации шифрования: крайний левый (старший) бит извлекается и все остальные значения сдвигаются влево, а правая ячейка (младший бит) заполняется значением функции обратной связи. Функция есть сложение (XOR) некоторых битов регистра. Для ее записи используется многочлен, у которого степень аргумента с ненулевыми коэффициентами указывает на номера подлежащих сложению бит. Важнейший показатель для LFSR – период псевдослучайной последовательности, который принимает возможный максимум $2^n - 1$, если многочлен функции обратной связи примитивен по модулю 2.

Сдвиг в регистрах LFSR происходит при выполнении определенного условия. Каждый LFSR содержит так называемый "бит управления тактированием": LFSR1 8-й бит, LFSR2 и LFSR3 – 10-й бит. На каждом шаге сдвигаются только те регистры у которых значение бита синхронизации совпадает с большинством значений синхронизирующих битов всех трех регистров.

Алгоритм начинается с инициализации регистров нулями:

$$\text{LFSR1} = \text{LFSR2} = \text{LFSR3} = 0. \quad (4.7)$$

На вход алгоритма поступают сеансовый ключ kc (64 бита), сформированный алгоритмом А8. Соответственно, имеем 64 повтора:

для $i = 0, 1, \dots, 63$:

$$\left\{ \begin{array}{l} \text{LFSR1}_0 = \text{LFSR1}_0 \oplus kc_i, \\ \text{LFSR2}_0 = \text{LFSR2}_0 \oplus kc_i, \\ \text{LFSR3}_0 = \text{LFSR3}_0 \oplus kc_i, \\ \text{Shift}(\text{LFSR1}), \\ \text{Shift}(\text{LFSR2}), \\ \text{Shift}(\text{LFSR3}). \end{array} \right. \quad (4.8)$$

Далее поступивший на вход номер кадра (22 бита) тоже обрабатывается итеративно:

для $i = 0, 1, \dots, 21$:

$$\left\{ \begin{array}{l} \text{LFSR1}_0 = \text{LFSR1}_0 \oplus \text{FrameCount}_i, \\ \text{LFSR2}_0 = \text{LFSR2}_0 \oplus \text{FrameCount}_i, \\ \text{LFSR3}_0 = \text{LFSR3}_0 \oplus \text{FrameCount}_i, \\ \text{Shift}(\text{LFSR1}), \\ \text{Shift}(\text{LFSR2}), \\ \text{Shift}(\text{LFSR3}). \end{array} \right. \quad (4.9)$$

Потом:

для $i = 0, 1, \dots, 99$:

$$\left\{ \begin{array}{l} M = \text{Majority}(\text{LFSR1}_8, \text{LFSR2}_{10}, \text{LFSR3}_{10}), \\ \text{Shift}(\text{LFSR1}, M), \\ \text{Shift}(\text{LFSR2}, M), \\ \text{Shift}(\text{LFSR3}, M). \end{array} \right. \quad (4.10)$$

Здесь *FrameCount* – 32-битная запись номера текущего фрейма. Здесь *Shift(LFSR)* – функция сдвига регистров на одну позицию с игнорированием битов синхронизации. Функция *Shift(LFSR, M)* сдвигает регистры на одну позицию с учетом битов синхронизации *M* (если *M* совпадает с битом синхронизации, то сдвиг происходит, иначе – нет. То есть сдвигаются только те регистры, синхробит которых принадлежит большинству.)

Функция *Majority(x, y, z)* вычисляется как

$$Majority(x, y, z) = (x \wedge y) \vee (x \wedge z) \vee (z \wedge y). \quad (4.11)$$

Генератор ключей создает ключевой поток в один бит при каждом тактовом импульсе. Прежде чем ключ создан, вычисляется мажоритарная функция. Затем каждый линейный регистр сдвига синхронизируется, если его бит синхронизации соответствует результату мажоритарной функции; иначе – он не синхронизируется.

Далее вычисляются 228 бит выходной последовательности. Половиной (114 бит) шифруются данные, исходящие из сети к мобильному телефону, другой половиной (114 бит) шифруются данные, идущие от телефона к сети. Разбивки по 114 бит называют кадрами. Схема шифра – простая гамма сложения по модулю два (рисунок 4.3). После передачи данных номер фрейма увеличивается на единицу и заново производится инициализация регистров.

Проблемы безопасности A5/1.

Еще в 2000 году было показано, что атака в реальном масштабе времени находит ключ за несколько минут на основе известных малых исходных текстов, но это требует этапа предварительной обработки с 248 шагами. В 2003 была опубликована атака, которая вскрывала A5/1 за несколько минут, используя анализ исходного текста.

Вообще, на сегодняшний день существует довольно много успешных атак на GSM-шифрование, которые относятся к атакам типа «*known-*

plaintext». То есть возможно восстановление ключа атакующим, если известны зашифрованные и незашифрованные данные, соответствующие друг другу. В стандарте GSM, помимо голосового трафика передаются еще и системные сообщения. Системные сообщения GSM содержат повторяющиеся данные и могут использоваться злоумышленником. В частности в 2010 году был предложен метод, основанный на поиске такого рода данных в шифротексте и простом переборе различных вариантов ключей, хранящихся в радужных таблицах, до тех пор пока не будет найден ключ, порождающий нужный шифротекст для известного заранее системного сообщения.

Рассмотрим одну из реализаций атак, предложенную в 2002 году. Реализуем так называемую корреляционную атаку.

Из определения процедуры инициализации (4.7) – (4.10) можно заключить, что начальное состояние регистров является линейной функцией от сессионного ключа kc и номера фрейма *FrameCount*.

Выходной бит генератора является XOR-ом выходных бит всех трех регистров, потому:

$$x = s_i^1 \oplus s_i^2 \oplus s_i^3 \oplus f_i^1 \oplus f_i^2 \oplus f_i^3. \quad (4.12)$$

Здесь s_i – последовательность, генерируемая регистрами после загрузки в них только битов ключа kc . f_i – последовательность, после загрузки только битов номера фрейма, x – выходной бит регистра.

Первые 100 циклов инициализации не производит никаких выходных битов. Первый бит выходной последовательности является 101-м сгенерированным битом. Таким образом, если учесть, что на каждом шаге вероятность сдвига для каждого регистра равняется $3/4$, можно сделать предположение, что после 101 шага, каждый регистр сдвигался ровно 76 раз. Следовательно, формула (4.12) преобразуется в:

$$s_{76}^1 \oplus s_{76}^2 \oplus s_{76}^3 = x_1 \oplus f_{76}^1 \oplus f_{76}^2 \oplus f_{76}^3. \quad (4.13)$$

Введя обозначение $O_{(76,76,76,1)}^j = x_1 \oplus f_{76}^1 \oplus f_{76}^2 \oplus f_{76}^3$, получим:

$$s_{76}^1 \oplus s_{76}^2 \oplus s_{76}^3 = O_{(76,76,76,1)}^j. \quad (4.14)$$

Поскольку выражение в правой части (4.14) нам известно, мы получаем 1 бит информации о ключевой последовательности s , а именно о состоянии 76-й позиции каждого регистра после инициализации. Действуя подобным образом, мы можем предположить, что на 102-й позиции LFSR1 остался также на позиции 76, а регистры LFSR2 и LFSR3 сдвинулись на позицию 77, таким образом, мы получаем информацию о 77-й позиции регистра, и так далее. Всего нам необходимо вскрыть 64 бита, для успешного восстановления начального состояния.

Ситуация (76,76,76) возникает именно на 101 шаге с крайне низкой вероятностью, и если бы мы решили действовать подобным образом, то нам понадобилось бы перебрать огромное количество фреймов, пока наконец не попадется именно тот, в котором после 101 сдвига каждый регистр прокрутился на 76 позиций. Для того чтобы уменьшить необходимое количество фреймов, был предложен следующий метод.

Не нужно угадывать конкретную позицию, в которой регистры проворачиваются (cl_1, cl_2, cl_3) раз. Достаточно просто знать, что с большой долей вероятности каждый регистр провернется от 76 до 102 на отрезке $I = [100, 140]$ выходных бит каждого фрейма.

Теперь для каждого фрейма можно вычислить вероятность:

$$\begin{aligned} & P(s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3) = 0) = \\ & = \sum_{t \in I} P((cl_1, cl_2, cl_3) \text{ в позиции } t) \times \left(O_{(cl_1, cl_2, cl_3, 100-t)}^j \right) + \\ & + \frac{1}{2} \left(1 - \sum_{t \in I} P((cl_1, cl_2, cl_3) \text{ в позиции } t) \right), \end{aligned} \quad (4.15)$$

где

$$P((cl_1, cl_2, cl_3) \text{ в позиции } t) = \frac{C_t^{t-cl_1} \times C_{t-(t-cl_1)}^{t-cl_2} \times C_{t-(t-cl_1)-(t-cl_2)}^{t-cl_3}}{4^t} \quad (4.16)$$

и обозначает вероятность того, что t -й бит был порожден позициями регистров (cl_1, cl_2, cl_3) .

Вычислив (4.15) для каждого доступного фрейма усредним полученные вероятности при помощи логарифма:

$$\Delta_{(cl_1, cl_2, cl_3)} = \sum_{j=1}^m \ln \left(\frac{P^j(s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3))}{1 - P^j(s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3))} \right). \quad (4.17)$$

Если $\Delta_{(...)}$ больше нуля, то $s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3) = 0$, в противном случае $s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3) = 1$.

Итак, вся атака полностью в виде алгоритма:

1. Выбираем интервал C . Например $C = [79, 86]$.
2. Пусть переменные cl_1, cl_2, cl_3 пробегают все значения из интервала C , для каждого фрейма вычисляем (4.15).
3. Для всех полученных значений вычисляем (4.17).
4. На основании значения $\Delta_{(...)}$ выбираем значение $s_1(cl_1) \oplus s_2(cl_2) \oplus s_3(cl_3)$.

Результатом выполнения данного алгоритма будут 512 уравнений вида $s_1(79) \oplus s_2(79) \oplus s_3(79) = 0$, состоящие из 8 неизвестных. Решив эту систему уравнений простым перебором, мы получим по 8 бит начального значения каждого регистра.

Повторив алгоритм еще два раза для интервалов $[87, 94]$ и $[95, 102]$ мы получим по 24 бита начального состояния каждого из регистров. Этого достаточно. Прокрутим каждый из регистров 101 раз назад мы получим то состояние регистров, которое было после второго шага инициализации, то есть после загрузки в регистры ключевых битов. И теперь мы можем сгенерировать всю ключевую последовательность целиком.

Описанная атака является одной из первых подобных атак. Наиболее совершенная из них позволяет с 90% вероятностью вскрыть сеансовый ключ из всего 2000 фреймов(9 секунд разговора). Таким образом, корреляционные атаки – весьма серьезная угроза не только в теории, но и на практике.

Далее отметим еще несколько моментов.

В алгоритме A5/1 серьезно нужно решать проблему управления ключами. Если в информационном обмене участвуют только Алиса и Боб, то им достаточно одного секретного ключа для использования симметричного шифра. Если участников становится n и каждый хочет иметь связь с каждым, то необходимо $n(n-1)/2$ секретных ключей.. То есть квадратичный рост таблиц шифрования в зависимости от числа участников сети. Поскольку для больших сетей хранить все эти ключи нереально, то были найдены альтернативные решения. Первое: каждый раз, когда Алиса и Боб хотят связаться, они могут создать между собой сеансовый (временный) ключ. Второе: могут быть установлены один или более центров распределения ключей, чтобы распределять сеансовые ключи для объектов. Все эти решения – часть *теории управления ключами*.

Еще одна проблема в симметричном шифровании – безопасная генерация ключа. Различные шифры нуждаются в ключах различных размеров. Если Алиса и Боб генерируют сеансовые ключи между собой, они должны выбрать ключ случайным образом, так, чтобы Ева не могла предвидеть, каков будет следующий ключ. Если ключи распределяет ключевой центр, то они должны иметь случайный характер, чтобы Ева не могла получить ключ, назначенный Алисе и Бобу, из ключа, назначенного Крэйгу и Еве.

СПИСОК РЕКОМЕНДУЕМОЙ И ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Басалова Г. Основы криптографии [Электронный ресурс]: электронный курс. URL: <http://www.intuit.ru/studies/courses/691/547/info>
2. Бехроуз А. Математика криптографии и теория шифрования [Электронный ресурс]: электронный курс. URL: <http://www.intuit.ru/studies/courses/552/408/info>
3. Введение в криптографию. Новые математические дисциплины: учебник / под ред. В. В. Ященко. – СПб.: Питер, 2001. – 288 с.
4. Гатченко Н.А., Исаев А.С., Яковлев А. Д. Криптографическая защита информации. – СПб.: НИУ ИТМО, 2012. – 142 с.
5. Иванов В. Криптография и шифрование [Электронный ресурс]: сборник видеолекций. URL: <https://events.yandex.ru/lib/talks/2337/>
6. Категория «Криптография» на Википедии [Электронный ресурс]: сайт. URL: [https://ru.wikipedia.org/wiki/ Категория : Криптография](https://ru.wikipedia.org/wiki/Категория:_Криптография)
7. Лапониная О.Р. Основы сетевой безопасности: криптографические алгоритмы и протоколы взаимодействия / О.Р. Лапониная. – М.: ИНТУИТ, 2005. – 608с.
8. Осипян В.О., Осипян К.В. Криптография в задачах и упражнениях. – М.: Гелиос АРВ, 2004 – 144 с.

9. Раздел по информационной безопасности на Хабрахабре [Электронный ресурс]: сайт. URL: <http://habrahabr.ru/hub/infosecurity/>
10. Раздел по криптографии на Хабрахабре [Электронный ресурс]: сайт. URL: <http://habrahabr.ru/hub/crypto/>
11. Скрипник Д. Общие вопросы технической защиты информации [Электронный ресурс]: электронный курс. URL: <http://www.intuit.ru/studies/courses/2291/591/info>
12. Фороузан Б.А. Криптография и безопасность сетей / Б.А. Фороузан. – М.: БИНОМ, 2010. - 784 с.
13. Цуцулис А. Сборник статей по криптографии [Электронный ресурс]: сайт. URL: <http://habrahabr.ru/users/neverwalkaloner/topics/>
14. Boneh D. Cryptography I [Электронный ресурс]: электронный интерактивный курс. URL: <https://www.coursera.org/course/crypto>
15. Boneh D. Cryptography II [Электронный ресурс]: электронный интерактивный курс. URL: <https://www.coursera.org/course/crypto2>
16. Cybersecurity Specialization University of Maryland [Электронный ресурс]: электронный интерактивный курс. URL: www.coursera.org/specialization/cybersecurity/7
17. Evans D. Applied Cryptography [Электронный ресурс]: электронный интерактивный курс. URL: <https://www.udacity.com/course/cs387>

18. Journey into cryptography [Электронный ресурс]: электронный курс. URL : <https://www.khanacademy.org/computing/computer-science/cryptography>

19. Zafar H., Green A. Cybersecurity and Its Ten Domains [Электронный ресурс]: электронный интерактивный курс. URL: <https://www.coursera.org/learn/cyber-security-domain/>

Учебное издание

Леонид Геннадьевич **Акулов**
Вадим Юрьевич **Наумов**

**ХРАНЕНИЕ И ЗАЩИТА
КОМПЬЮТЕРНОЙ ИНФОРМАЦИИ**

Учебное пособие

Редактор *Л. Н. Рыжих*

Темплан 2015 г. (учебники и учебные пособия). Поз. № 27.
Подписано в печать 23.04.2015. Формат 60х84 1/16. Бумага газетная.
Гарнитура Times. Печать офсетная. Усл. печ. л. 3,72. Уч.-изд. л. 2,78.
Тираж 100 экз. Заказ

Волгоградский государственный технический университет.
400005, г. Волгоград, просп. им. В. И. Ленина, 28, корп. 1.

Отпечатано в типографии ИУНЛ ВолгГТУ.
400005, г. Волгоград, просп. им. В. И. Ленина, 28, корп. 7.